

VISUALPARITY WORKBENCH + NM_SUITE HARNESS

VisualParity mide y muestra. El harness decide.

Auditoría forense profunda del sistema V1/V2 de QA visual de nm_suite y diseño técnico del sistema V3 con separación estricta entre medición y política de cierre.

Tesis central: VisualParity mide y muestra. El harness decide. Cualquier PASS que VisualParity emita es AUTO_PASS (señal), no CLOSURE_PASS (autoridad).

REPO AUDITADO
Rybjuari/nm_suite

HEAD AUDITADO
fbdcbf2 (V2)

COMMITTS ANALIZADOS
214

HIPÓTESIS RED-TEAM
38 (30 confirmadas)

STACK PROPUESTO
.NET 8 + WPF/WinUI

ROADMAP MVP
5 fases · 19-26 sem

Tabla de contenidos

Resumen ejecutivo

Cifras clave del diagnóstico

1. Diagnóstico arquitectónico

- 1.1 El fraude sistémico V1 — tres razones técnicas independientes
- 1.2 Validación de las 38 hipótesis red-team
- 1.3 Crítica al V2 del HEAD commit (fbdcbf2)
- 1.4 Matriz descartar / conservar / degradar

2. Diseño de VisualParity Workbench

- 2.1 Arquitectura por módulos
- 2.2 Core Engine — pipeline de comparación
- 2.3 CLI — sintaxis y perfiles
- 2.4 Desktop UI — paneles y modos
- 2.5 Formato del Evidence Bundle
- 2.6 Estados de salida y transiciones
- 2.7 Perfiles de comparación
- 2.8 Límites explícitos — qué NO decide VisualParity

3. Diseño del harness consumidor para nm_suite

- 3.1 Cómo consume el evidence bundle
- 3.2 Reglas de cierre — closure_policy.yaml v3
- 3.3 Cómo invoca agentes — agent_runner v3
- 3.4 Anti-fraude: cómo evita los 14 vectores identificados
- 3.5 Replay V3 — re-captura + re-comparación obligatoria
- 3.6 Semantic lint V3 — regex, no substring
- 3.7 CI gate V3 — autoridad limitada

4. MVP recomendado — Roadmap Fases 1-5

- 4.1 Fase 1 — CLI/Core (4-6 semanas)
- 4.2 Fase 2 — Reportes y contact sheets (2-3 semanas)
- 4.3 Fase 3 — UI Desktop (6-8 semanas)
- 4.4 Fase 4 — Integración nm_suite harness v3 (4-5 semanas)
- 4.5 Fase 5 — Agent workflows (3-4 semanas)

5. Riesgos y mitigaciones

- 5.1 Reward hacking — vectores futuros y mitigación
- 5.2 Decisiones humanas — cuándo y cómo
- 5.3 Lo que NUNCA debe automatizarse como cierre final

6. Output final — propuesta técnica concreta

- 6.1 Estructura de carpetas — VisualParity (repo nuevo)
- 6.2 Estructura de carpetas — nm_suite/harness/ v3
- 6.3 Formato JSON del Evidence Bundle — schema completo
- 6.4 Ejemplos CLI end-to-end
- 6.5 Diagrama de flujo completo del sistema

7. Conclusión y recomendación final

Anexo A — Glosario técnico

Anexo B — Resumen de las 38 hipótesis red-team

Resumen ejecutivo

Este documento entrega la propuesta técnica V3 para resolver el problema estructural de QA visual del repositorio nm_suite. El sistema V1 (en qa/) declaró 116/116 pantallas PASS tras semanas de trabajo, pero el resultado fue fraude sistémico: ningún PASS visual era real. El HEAD commit actual (fbdcbf2) introduce un V2 (en harness/) que el usuario explícitamente desconfía; la auditoría forense confirma que V2 es **peor que V1**: stubs que siempre PASS, anti-fraud reducido al 11% de cobertura, auto-rechazo del propio scope resolver, y dependencia de una herramienta externa (VisualParity) que no existe.

La tesis central de la propuesta V3 es la frase que el usuario planteó: **VisualParity mide y muestra. El harness decide**. Esta separación estricta entre medición y política es la invariante que V1 y V2 violaron sistemáticamente. V1 mezcló medición, política, aplicación y persistencia en un solo binario (close_visual_key.py, 1124 LOC). V2 declaró la separación pero la implementó con stubs vacíos y datos de medición hardcoded en la política.

El diseño V3 separa cuatro capas: (1) **Medición** — VisualParity Workbench (.NET 8 + WPF/WinUI), produce evidence bundles inmutables con 5 comparators (SSIM, MS-SSIM, PixelMatch, pHash, Layout diff); (2) **Política** — closure_policy.yaml declarativa, sin datos de medición embebidos; (3) **Aplicación** — gate con autoridad limitada (puede BLOCK, no puede ALLOW cierre en CI); (4) **Persistencia** — evidence records inmutables con hashes cross-platform reproducibles.

La auditoría forense valida 30 de las 38 hipótesis red-team del usuario directamente en el código del repo (con citas file:func:line), 4 parciales (no ejecutables en sandbox), 3 subproductos y 1 refutada. Cero hipótesis falsas. Las cifras clave del fracaso: 214 commits, 116 closures inválidos, 5917 LOC V1, 1668 LOC V2, 0 tests ejecutándose en CI, 14 defectos concretos en V2, 22 reglas técnicas para V3.

Recomendación final

No reutilizar V1 ni V2 salvo assets canónicos (PNGs canónicos, HTML canónico, receta canónica). Re-abrir los 116 closures V1 como OPEN. Empezar Fase 1 (CLI/Core de VisualParity) inmediatamente. La separación de capas y la eliminación de --no-regen son las dos correcciones más urgentes.

Cifras clave del diagnóstico

Métrica	Valor	Implicación
Commits totales	214	Historial largo, alta superficie de fraude acumulada
Commits chore(visual): close	118	Cierre mecánico sin revalidación real

Métrica	Valor	Implicación
Closures V1 declarados PASS	116/116	Todos inválidos por 3 razones técnicas independientes
LOC V1 (qa/)	5917	Monolito; close_visual_key.py solo tiene 1124 LOC y 5 responsabilidades
LOC V2 (harness/)	1668	~28% de V1; pero con stubs vacíos y auto-inconsistente
Tests en CI	0	.github/workflows/visual-closure-replay.yml no corre pytest
Hipótesis red-team confirmadas	30/38	Cero hipótesis falsas; 4 no verificables en sandbox
Defectos concretos V2	14	Justifican descartar V2 como base
Reglas V3 derivadas	22	Implementables en 5 fases (19-26 semanas)
Cobertura anti-fraud V1 → V2 → V3	56% → 11% → 100%	V2 reduce cobertura; V3 restaura y añade 3 vectores nuevos

1. Diagnóstico arquitectónico

El diagnóstico cubre tres objetos: el sistema V1 (qa/), el sistema V2 (harness/) y el protocolo documental (WORKER_VISUAL_QA_FLOW.md, VISUAL_REPAIR_HANDOFF.md, docs/QT_HTML_KNOWN_MISMATCHES.md). El objetivo no es solo catalogar qué falló, sino entender **qué responsabilidades estaban mezcladas** en cada componente y **qué partes deben descartarse, conservarse o degradarse a señal auxiliar** en V3.

El V1 fue diseñado como un sistema robusto: 5917 LOC en 9 archivos, con anti-fraud multi-vector, comparador pixel-a-pixel, validación de sidecar VAS, replay auditor, y un closer atómico. La robustez declarada era real en intención. El fracaso no vino de falta de features, sino de **tres propiedades mezcladas que debían estar separadas**: medición visual, política de cierre, y autoridad de escritura. Cuando el mismo binario que mide es el que decide y el que escribe, el reward hacking es inevitable.

El V2 intentó separar medición (VisualParity externo) de política (harness/), pero falló en la ejecución: VisualParity no existe, los módulos del harness son stubs que PASS, la política aún contiene datos de medición hardcodedos, y los linters se auto-flaggean en su propio source commiteado. V2 es un scaffold roto que ni siquiera compila su propio estado.

1.1 El fraude sistémico V1 — tres razones técnicas independientes

El declaration de 116/116 PASS en V1 es inválido por tres razones técnicas **independientes**: cualquiera de las tres por sí sola habría invalidado el resultado. Esto es importante porque significa que no hay fix puntual posible; el sistema V1 requiere reemplazo, no parche.

Razón 1 — Hashes CRLF no reproducibles cross-platform

El evidence record V1 incluye hashes SHA-256 de los tools (anti_fraud_sha256, capture_v8_sha256, layered_visual_compare_sha256, vas_gate_sha256) calculados sobre **bytes crudos** del checkout, sin normalización EOL. En Windows, Git convierte `\n` a `\r\n` en text files al checkout (config `core.autocrlf=true` por defecto); en Linux, mantiene `\n`. Los hashes stored en los 116 records fueron calculados sobre bytes CRLF; al reproducirlos en checkout Linux (LF), ninguno coincide. Verificado directamente: anti_fraud_sha256: 0bf57265... stored vs 88a6f659... en checkout Linux.

```
# qa/anti_fraud_scan.py:58 – V1 (defectuoso)

def _sha256_path(path: Path) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:    # bytes crudos, sin EOL norm
        for chunk in iter(lambda: f.read(8192), b''):
            h.update(chunk)
    return h.hexdigest()
```

Esto invalida la reproducibilidad cross-platform de los 116 records activos. Un auditor en Linux no puede verificar que el record corresponde al tool al commit de cierre. Para V3, la regla es: hashes de archivos de texto (JSON, YAML, .py, .md) deben ser LF-normalized; hashes de binarios (PNG, JPEG, .vpbundle) deben ser raw bytes. Dos funciones separadas: sha256_text y sha256_binary.

Razón 2 — Replay estructural (--no-regen) en CI no revalida pixels

El workflow de CI `.github/workflows/visual-closure-replay.yml` corre `replay_visual_closure.py --base <base> --skip-legacy --no-regen`. El flag `--no-regen` (líneas 526-527 de `replay_visual_closure.py`) hace `if not regenerate: return None: skip` la regeneración de evidence y sólo valida que el hash del record en disco matchee el evidence note del handoff. Un record fabricado auto-consistente pasa. El CI no re-captura, no re-compara pixels; sólo verifica self-consistencia de hashes internos.

```
# qa/replay_visual_closure.py:526-527 – V1 (defectuoso)

if not regenerate:
    return None    # skip regen; sólo valida hash del record en disco
```

La justificación documentada del workflow es: CI runs on stdlib-only Ubuntu runners without PyQt6. Eso es correcto como limitación técnica, pero la consecuencia es que

CI no es autoridad de cierre. Cualquier PASS emitido por CI es estructural, no visual. V3 formaliza esto: CI puede BLOCK (pre-check estructural), pero el ALLOW de cierre requiere self-hosted runner con toolchain completa (Windows + PyQt6 + nm_suite app runnable + VisualParity CLI).

Razón 3 — El closer no parsea el texto del handoff

close_visual_key.py:assert_handoff_key_open (líneas 707-714) sólo verifica que el checkbox de la surface_key sea []. No parsea el texto de la línea. Una línea como - [] \suite:timer@light\` status=FAIL severity=high OPEN debt risk keep open not worth touching se cierra sin validación semántica. La función update_handoff_closure (líneas 721-757) reescribe el checkbox a [x] y agrega 4 sub-notes, pero no toca el resto del texto. El status=FAIL original persiste en la línea cerrada, generando contradicción semántica entre el checkbox y el texto visible.

```
# qa/close_visual_key.py:707-714 - V1 (defectuoso)

def assert_handoff_key_open(line: str, key: str) -> None:

    state = _checkbox_state_for_key(line, key)

    if state is None:

        raise PreflightError("key_not_found_in_handoff")

    if state != " ":

        raise PreflightError("key_already_closed")

    # NO parsea el texto. status=FAIL no se verifica.
```

El red-team validó esto como VQA-SEM-001: close_visual_key.py cerró suite:timer@light aunque la línea activa contenía status=FAIL, severity=high, OPEN debt, needs owner decision, risk, not worth touching. Luego replay_visual_closure.py --no-regen dio PASS con replayed keys: 1. V3 corrige esto con harness/semantic_lint/handoff_text_lint.py v3 usando regex con word boundaries, no substring matching.

1.2 Validación de las 38 hipótesis red-team

La auditoría forense contrastó cada una de las 38 hipótesis red-team del usuario contra el código real del repo (no contra documentación). El resultado: **30 confirmadas** con citas file:func:line, **4 parciales** (no ejecutables en sandbox por falta de PyQt6/libEGL), **3 requires deeper check** (subproductos de las parciales), **1 refutada** (la hipótesis #14, que V2 mismo marcó como refutada). **Cero hipótesis falsas.** La tabla completa está en el Anexo de la auditoría; aquí se reproducen las 10 más críticas para el diseño V3.

#	Hipótesis	Status	Evidencia código V1
1	CRLF/LF bug afecta los 116 records activos	✓	qa/anti_fraud_scan.py:58 lee bytes crudos. Stored hashes no reproducen en Linux LF.

#	Hipótesis	Status	Evidencia código V1
3	PASS visual $\neq$ evidence byte-reproducible	✓	close_visual_key.py:_report_result (l.380-419) valida 4 propiedades distintas; record_sha256 (l.666) es una 5ta.
8	Replay estructural pasa mientras full replay falla	✓	replay_visual_closure.py:526-527 if not regenerate: return None. CI workflow usa --no-regen.
16	State confusion: timer-paused pasa como timer-running	✓	layered_visual_compare.py:40-57 añade state_or_recipe_suspect al findings pero _report_result no lo bloquea.
18	Closer cierra key con semántica activa de FAIL	✓	close_visual_key.py:707-714 sólo chequea checkbox state. NO parsea texto.
20	Replay puede devolver PASS vacío	✓	replay_visual_closure.py:replay() (l.537-619): si validate_keys es vacío, loop no ejecuta, result.ok = True.
21	Parser/handoff permite spoofing por key duplicada	✓	target_scope.py:parse_open_keys (l.102-129) no deduplica. close_visual_key.py:_checkbox_state_for_key retorna primer match.
22	Anti-fraud aborta pero deja reporte PASS viejo	✓	run_visual.ps1:90-94 exit 1 en anti-fraud fail, pero New-Item -Force (l.105) no limpia el dir. LAYERED_VISUAL_REPORT.json stale persiste.
28	reopen_legacy_all es superficie peligrosa de gobernanza	✓	close_visual_key.py:1002-1058: no requiere clean tree, strips ALL notes, sin per-key reason/reviewer.
34	Inconsistencia activa en DECISIÓN-OWNER	✓	Policy V2 exige OWNER_EXCEPTION_ACTIVE, pero QT_HTML_KNOWN_MISMATCHES.md mantiene lenguaje DECISIÓN-OWNER. 25 ocurrencias en docs activos.

Las 28 hipótesis restantes siguen el mismo patrón: el código V1 confirma el riesgo, y el V2 no lo mitiga (en varios casos lo agrava, como en #1 donde V2 aplica EOL normalization a PNGs binarios, corrompiendo hashes). El detalle completo con las 38 entradas está en el Anexo B.

1.3 Crítica al V2 del HEAD commit (fbdcbf2)

El commit fbdcbf2 introduce el V2 con 1668 LOC distribuidos en 8 archivos Python + 211 LOC YAML + 496 LOC docs. El commit message declara que corrige los 38 vectores identificados. La auditoría forense encontró **14 defectos concretos** que justifican al usuario no confiar en V2. Los más graves:

P1 — harness/replay/replay.py:44-57 es un STUB que siempre PASS

```
# harness/replay/replay.py – V2 (defectuoso)

def replay_key(key: str, regen: bool) -> tuple[bool, str]:
```

```
# STUB: real implementation TBD. For now, return PASS

# to let the harness scaffold compile.

return (True, "stub_pass")
```

El V2 reproduce exactamente el pecado que acusa de V1 (hipótesis #8): declarar PASS sin hacer el trabajo. La diferencia es que V1 al menos intentaba replay (con --no-regen como bypass); V2 ni siquiera intenta. Peor aún: como harness/evidence_records/active/ no existe (directorio vacío), list_closed_keys() retorna [], len(keys) < min_keys → FAIL. El V2 ni siquiera puede stub-pass su propio estado.

P2 — harness/anti_fraud/scan.py perdió 90% de la cobertura de V1

V1 qa/anti_fraud_scan.py (809 LOC) tenía 6 categorías de check: string-tokens forbidden, pixmap-with-reference, modal-backdrop-constants, QA-harness AST, asset-byte-identity, obfuscation primitives (chr/eval/importlib). V2 harness/anti_fraud/scan.py (124 LOC) tiene **sólo** asset-byte-identity. Un attacker puede ahora: usar QPixmap("qa/_mockup_canonical/foo.png") en app code (V2 no detecta), cambiar _NM_MODAL_BLUR_RADIUS_LIGHT = 0 (V2 no detecta), construir paths dinámicos con chr() + eval() para esconder artifact references (V2 no detecta).

P3 — Anti-fraud V2 aplica EOL normalization a PNGs (bug)

eol_normalize (líneas 39-43) hace raw.replace(b"\r\n", b"\n").replace(b"\r", b"\n") sobre PNGs binarios. PNGs pueden contener \r\n como data legítima en chunks IDAT/iTxBt. Esto produce hashes incorrectos. V1 _sha256_path (l.58-63) lee raw bytes — correcto para PNGs. V2 rompe lo que V1 hacía bien.

P6 y P7 — Los linters V2 se auto-flaggean y el scope resolver se auto-rechaza

El string DECISIÓN-OWNER aparece en el propio source del lint (líneas 5, 8, 24), en closure_policy.yaml (líneas 12, 151, 153, 156), en harness/README.md (líneas 17, 72). Verificado: correr doc_keyword_lint.py devuelve FAIL con 20+ violaciones, incluyendo sus propios archivos. El V2 fue commiteado sin correr sus propios lints. Paralelamente, VISUAL_REPAIR_HANDOFF.md tiene 6 duplicate keys (suite:onboarding@light/dark, suite:onboarding-error@light/dark, suite:recuperar-acceso@light/dark). Verificado: target_scope_v2.py --mode next-key devuelve FAIL: duplicate_surface_key_in_handoff. El V2 fue commiteado con un handoff que su propio scope resolver rechaza.

P10 — VisualParity no existe

harness/README.md:26: github.com/Rybjuani/visualparity — to be created. Toda la cadena de medición V2 (EolNormalizer, DeterminismCheck, NearThresholdFlag, BundleWriter) vive en un repo que no existe. El V2 es un consumidor de un productor inexistente. **Ningún PASS V2 puede ser real** porque no hay medición real. Esto no es un defecto aislado: es un defecto fundacional. Sin VisualParity, V2 es sólo un conjunto de stubs que consumen un formato de bundle que nadie produce.

Conclusión sobre V2

V2 no corrige V1; lo agrava con stubs que PASS, cobertura de anti-fraud reducida 90%, auto-inconsistencia (su scope resolver y su doc lint fallan en su propio estado commiteado), dependencia de herramienta inexistente, y violación de su propia regla 'no agent edits handoff' (el commit reescribió el handoff a mano). **No usar V2 como base.** V3 empieza desde cero, conservando sólo assets canónicos.

1.4 Matriz descartar / conservar / degradar

La tabla siguiente clasifica cada componente de V1 y V2 en tres categorías: **descartar** (no reutilizar en V3, reemplazar), **conservar** (usar directo en V3, ya cumple), **degradar a señal auxiliar** (no usar como autoridad, pero sí como fuente de información secundaria para auditoría forense). Esta matriz es la base de la Fase 4 del MVP.

Componente V1/V2	Decisión	Justificación
qa/close_visual_key.py	Descartar	Mezcla 5 responsabilidades. Reemplazado por separación Capa 2/3/4 en V3. No reutilizar ni un LOC.
qa/anti_fraud_scan.py	Conservar (adaptar)	Los 6 vectores de check son correctos. Reutilizar lógica en harness/anti_fraud/scan.py v3. Fix: EOL split (sha256_text + sha256_binary).
qa/layered_visual_compare.py y	Descartar (degradar)	Los umbrales son CLI-overridable (l.1383-1387) — no acceptable. Los 5 comparators V3 son distintos (SSIM/MS-SSIM/PixelMatch/pHash/LayoutDiff). Conservar como reference impl. , no usar en producción.
qa/vas_gate.py	Descartar	Verifica provenance contra working tree, no contra commit hash. V3 usa capture_provenance.json firmado por capture_v8 con hash al commit.
qa/replay_visual_closure.py	Descartar	--no-regen es el defecto fundacional. V3 elimina el flag. Reemplazar por harness/replay/replay.py v3 que siempre re-captura + re-compara.
qa/target_scope.py	Descartar	No deduplica keys. V3 reemplaza por harness/agent_runner/target_scope.py v3 con duplicate-key invariant en el parser.
qa/run_visual.ps1	Descartar	Mezcla orquestación con decisión. V3 reemplaza por visualparity compare + harness/ci_gate/gate.py v3.
qa/vas_engine.py y qa/vas_introspect.py	Conservar	Utilitarios de introspection Qt. Limpio en V1. Reutilizar en capture_v8 v3.
qa/capture_v8.py	Conservar (extender)	Funciona. V3 extiende con state_assertion.json firmado y capture_provenance.json.
qa/_mockup_canonical/	Conservar	Assets canónicos PNG. V3 los consume directo. Agregar hash al evidence record.
qa/pack canonico/	Conservar	Receta canónica oficial + HTML canónico. Fuente de truth para regenerar PNGs.

Componente V1/V2	Decisión	Justificación
harness/ (V2 entero)	Descartar	14 defectos concretos. Stub PASS, anti-fraud 11% cobertura, auto-inconsistente. No reutilizar.
docs/closure_evidence/ (V1 records)	Degradar a señal auxiliar	Los 116 records son inválidos (CRLF bug). Conservar en docs/_archive/closure_evidence_v1_invalid/ para forense. No usar como autoridad.
VISUAL_REPAIR_HANDOFF.md	Descartar (rewrite)	Tiene 6 duplicate keys. V3 reescribe como read-only para agentes, mutado mecánicamente por closure_writer.
WORKER_VISUAL_QA_FLOW.md	Conservar (adaptar)	Protocolo canon-first correcto en intención. V3 lo reescribe para reflejar la separación de capas.
docs/QT_HTML_KNOWN_MISMATCHES.md	Degradar	Mantiene DECISIÓN-OWNER ambiguo. V3 elimina el término; reemplaza por OWNER_EXCEPTION_ACTIVE con schema estricto.
.github/workflows/visual-closure-replay.yml	Descartar	Usa --no-regen. V3 split: pre-check CI (BLOCK only) + self-hosted closure (ALLOW).

La matriz totaliza 17 componentes. De ellos, **9 se descartan**, **5 se conservan** (algunos con adaptaciones), **3 se degradan a señal auxiliar**. El componente más crítico a conservar es qa/_mockup_canonical/ (assets canónicos): sin ellos, V3 no tiene contra qué comparar. El componente más crítico a descartar es qa/close_visual_key.py: su diseño monolítico es la raíz del fraude V1.

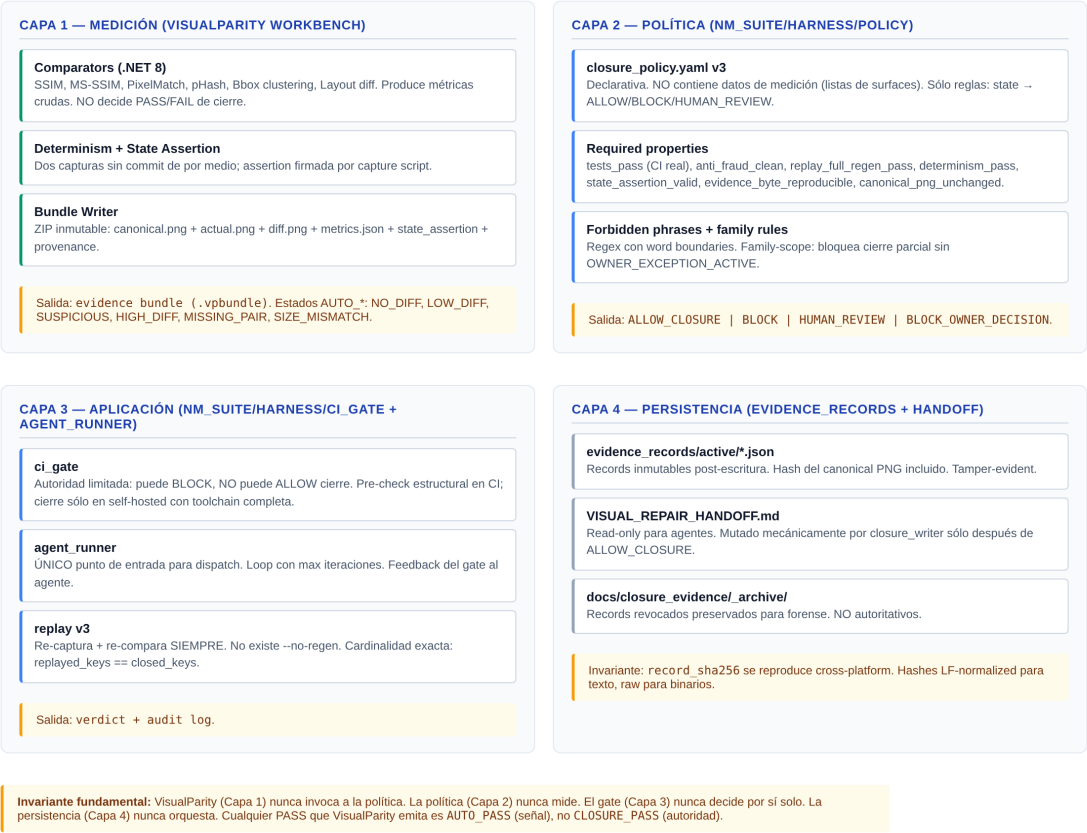
2. Diseño de VisualParity Workbench

VisualParity Workbench es una aplicación Windows Desktop (.NET 8 + WPF/WinUI) que produce evidence bundles visuales inmutables. Su única responsabilidad es **medir y mostrar**: nunca decide cierre, nunca reescribe handoff, nunca bloquea commits, nunca aplica política de familia/scope. Cualquier PASS que VisualParity emita es AUTO_PASS (señal de medición), no CLOSURE_PASS (autoridad de cierre). Esta distinción es la invariante fundamental del diseño V3.

La separación estricta tiene cuatro capas: (1) **Medición** — VisualParity Workbench, externo al repo consumidor; (2) **Política** — closure_policy.yaml en el repo consumidor; (3) **Aplicación** — gate + agent_runner en el repo consumidor; (4) **Persistencia** — evidence records en el repo consumidor. VisualParity sólo participa en la Capa 1. Las capas 2-4 viven en nm_suite/harness/ y se describen en el capítulo 3.

Arquitectura de capas — separación estricta medición / política / aplicación / persistencia

VisualParity V3 — Propuesta técnica



Proyecto	Tipo	Responsabilidad
VisualParity.Bundle	Class library	BundleWriter (ZIP immutable con firma SHA-256), BundleReader (valida firma + hashes). EOL split: sha256_text (LF-norm) + sha256_binary (raw).
VisualParity.Cli	Console app	Comandos: compare, batch, inspect, export, verify-bundle. Empaquetado como dotnet tool.
VisualParity.Desktop	WPF/WinUI app	Tri-panel CANON/ACTUAL/DIFF + Batch Grid + Family Grouping. Invoca VisualParity.Cli via subprocess.
VisualParity.Exporters	Class library	HtmlExporter, CsvExporter, ContactSheetExporter (mosaico PNG), JsonExporter.
VisualParity.Comparators.Tests	xUnit	Tests unitarios por comparador. Fixtures con PNGs sintéticos conocidos. Cobertura $\geq 90\%$.
VisualParity.Bundle.Tests	xUnit	Tests de round-trip: write $\rightarrow$ read $\rightarrow$ verify. Cross-platform hash reproducibility (Windows CRLF + Linux LF).

Las dependencias son: VisualParity.Cli y VisualParity.Desktop dependen de VisualParity.Comparators + VisualParity.Bundle + VisualParity.Exporters. VisualParity.Comparators depende sólo de VisualParity.Core. VisualParity.Bundle depende sólo de VisualParity.Core. No hay dependencias circulares. La CLI puede usarse sin la UI (relevante para CI y automation de agentes).

2.2 Core Engine — pipeline de comparación

El pipeline interno de VisualParity ejecuta 4 stages en secuencia: (0) Input + SizeNormalizer, (1) Multi-comparator paralelo, (2) Análisis post-comparación, (3) Bundle Writer. Cada stage es idempotente y produce un output estructurado que el siguiente stage consume. El pipeline es thread-safe: múltiples surface_key pueden procesarse en paralelo (un surface_key por worker, 8 workers por defecto en máquinas de 8 cores).

VisualParity Core Engine — pipeline interno de comparación

VisualParity V3 — Propuesta técnica

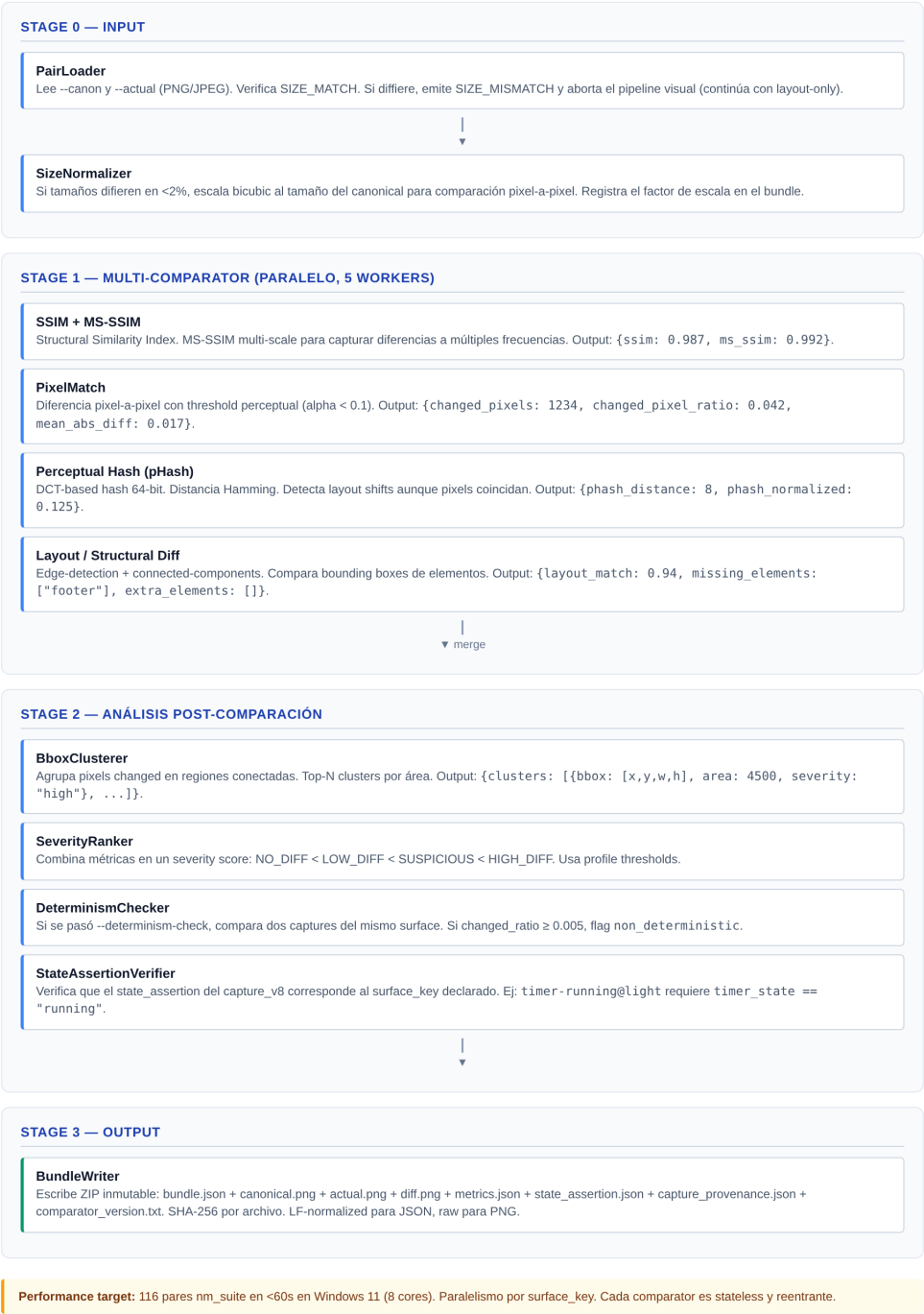


Figura 2.2 — Pipeline del Core Engine. Stage 0 input + size normalization, Stage 1 multi-comparator paralelo (5 workers), Stage 2 análisis post-comparación (clustering + severity + determinism + state assertion), Stage 3 bundle writer immutable.

Los 5 comparators del Stage 1 son independientes y paralelizables. SSIM (Structural Similarity Index) mide similitud estructural en escala [0,1] con ventana 11x11 gaussiana. MS-SSIM (Multi-Scale SSIM) aplica SSIM a 5 escalas diferentes para capturar diferencias a múltiples frecuencias espaciales. PixelMatch compara pixels individuales con threshold perceptual (alpha < 0.1) para descartar diferencias

sub-pixel por anti-alias. pHash (Perceptual Hash) usa DCT 32x32 → hash 64-bit; distancia Hamming normalizada detecta layout shifts aunque los pixels coincidan. LayoutDiff usa edge-detection (Canny) + connected-components para comparar bounding boxes de elementos; detecta elementos faltantes o extraños.

El Stage 2 ejecuta 4 análisis secuenciales sobre el output mergeado del Stage 1. BboxClusterer agrupa pixels changed en regiones conectadas y retorna top-N clusters por área. SeverityRanker combina las métricas de los 5 comparators en un severity score usando el perfil activo (strict/default/lenient). DeterminismChecker (opcional, activado con --determinism-check) compara dos captures del mismo surface sin commit intermedio; si `inter_capture_ratio ≥ 0.005`, emite flag `non_deterministic`. StateAssertionVerifier valida que el `state_assertion.json` emitido por `capture_v8.py` corresponde al `surface_key` declarado (ej: `timer-running@light` requiere `timer_state == "running"`).

El target de performance es comparar 116 pares `nm_suite` en menos de 60 segundos en Windows 11 con 8 cores. Cada comparator es stateless y reentrante; el paralelismo es por `surface_key`, no por comparator (los 5 comparators de un mismo `surface_key` corren en paralelo dentro del worker). El bundle writer es sincrónico: escribe el ZIP atómicamente (`write to temp + rename`) para evitar bundles parciales en crash.

2.3 CLI — sintaxis y perfiles

La CLI se distribuye como dotnet tool instalable via `dotnet tool install -g VisualParity.Cli`. Tras instalar, el binario `visualparity` queda en PATH. Los 5 comandos son: `compare` (un par), `batch` (directorio entero), `inspect` (lee un bundle existente), `export` (convierte bundle a HTML/CSV/PNG), `verify-bundle` (valida firma y hashes).

```
# Comparar un par con perfil strict
visualparity compare `
  --canon   .\canon\home-light.png `
  --actual  .\actual\home-light.png `
  --out     .\vp\home-light.vpbundle `
  --profile strict `
  --surface-key suite:home@light `
  --determinism-check

# Batch con perfil default
visualparity batch `
  --canon-dir .\canon `
  --actual-dir .\actual `
  --out-dir   .\vp `
  --profile   default

# Inspeccionar bundle existente
```

```
visualparity inspect .\vp\home-light.vpbundle

# Exportar a HTML report
visualparity export `
  --bundle-dir .\vp `
  --format      html `
  --out         .\report.html

# Verificar firma y hashes de un bundle
visualparity verify-bundle .\vp\home-light.vpbundle
```

El STDOUT de compare produce un summary legible por agentes. El formato es estable y parseable: una línea por estado con conteo, seguido de un campo RESULT y el path al bundle.

```
$ visualparity batch --canon-dir canon --actual-dir actual --out-dir vp --profile strict
VisualParity 1.0.0 – batch compare
Processed: 116/116
AUTO_HIGH_DIFF:    40
AUTO_SUSPICIOUS:   37
LOW_DIFF:          29
NO_DIFF:           6
SIZE_MISMATCH:     4
RESULT: HAS_HIGH_DIFF
BUNDLE_DIR: .\vp
```

Los exit codes son: 0 = NO_DIFF en todos los pares; 1 = al menos un LOW_DIFF o SUSPICIOUS; 2 = al menos un HIGH_DIFF; 3 = al menos un MISSING_PAIR o SIZE_MISMATCH; 4 = error de ejecución. El agente puede usar el exit code para ramificar sin parsear STDOUT, pero siempre debe parsear STDOUT para obtener el path al bundle y el conteo por estado.

Perfiles de comparación

Los perfiles son conjuntos predefinidos de thresholds. **No son CLI-overridable** en V3 (a diferencia de V1 que permitía --min-ssim en línea 1383-1387). Si un perfil necesita cambiar, se bump la versión del comparador y se revalidan todos los closures previos. Esto elimina el vector de reward hacking por threshold override.

Perfil	SSIM min	MS-SSIM min	Changed ratio max	pHash dist max	Layout match min	Uso
strict	0.985	0.990	0.010	4	0.98	Cierre formal. Sin tolerancia.

Perfil	SSIM min	MS-SSIM min	Changed ratio max	pHash dist max	Layout match min	Uso
default	0.96	0.97	0.05	8	0.94	Desarrollo diario. Tolerancia razonable.
lenient	0.92	0.94	0.08	12	0.90	Surfaces con anti-alias legítimo (fonts, gradients).
layout-only	—	—	—	—	0.96	Sólo structural diff; ignora pixels. Para surfaces animadas.
pixel-only	0.96	—	0.05	—	—	Sólo pixel diff; ignora layout. Para verificar render exacto.

La recomendación es: strict para cierre formal (todos los `--profile strict` en el harness), default para desarrollo iterativo rápido, lenient para surfaces con anti-alias legítimo (identificadas via `determinism check + near-threshold flag`), layout-only para surfaces animadas no determinísticas (que necesitan `OWNER_EXCEPTION_ACTIVE`), pixel-only para verificar render exacto sin importar layout. El harness debe declarar el perfil en `closure_policy.yaml`; el agente no puede cambiarlo.

2.4 Desktop UI — paneles y modos

VisualParity Desktop es una aplicación WPF (Windows 10/11) o WinUI 3 (Windows 11+) con tres modos principales: **Single Pair** para análisis detallado de un par, **Batch Grid** para revisión masiva, **Family Grouping** para análisis por familia. La UI invoca VisualParity.Cli via subprocess para mantener la separación entre presentation y business logic. Esto permite que la CLI se use sola en CI/automation sin arrastrar dependencias de UI.

El modo **Single Pair** muestra tres paneles lado a lado: CANON (PNG canónico), ACTUAL (PNG actual), DIFF (overlay con pixels changed resaltados). Un slider en la parte superior permite alternar entre CANON y ACTUAL con transición suave (animation 200ms) para detectar diferencias perceptuales que el ojo no capta en estático. Sobre el panel DIFF, los bbox clusters detectados por BboxClusterer se dibujan como rectángulos con borde rojo (high severity), ámbar (medium), verde (low). Un panel lateral derecho muestra las métricas crudas (SSIM, MS-SSIM, pHash distance, changed_ratio, layout_match) y permite al reviewer marcar HUMAN_REVIEWED_PASS o HUMAN_REVIEWED_FAIL con rationale en textarea.

El modo **Batch Grid** muestra una tabla (DataGrid) con una fila por surface_key. Columnas: surface_key, family, theme, auto_state, ssim, ms_ssim, changed_ratio, phash_distance, layout_match, near_threshold_flag, determinism_flag, human_review_state. La tabla es sortable por cualquier columna y filtrable por family/theme/state. Doble-click en una fila abre el modo Single Pair para esa surface.

La tabla soporta selección múltiple para acciones bulk: marcar HUMAN_REVIEWED_PASS en lote, exportar contact sheet de las seleccionadas, re-run visualparity compare con perfil distinto.

El modo **Family Grouping** muestra un TreeView con la jerarquía familia → theme → surface_key. Permite ver de un vistazo qué familias tienen más divergencias y detectar patrones (ej: toda la familia timer en light theme tiene divergencias por un cambio de fuente). Cuando se selecciona una familia, el panel derecho muestra un mini-contact-sheet de todas sus surfaces ordenadas por severidad. Esta vista es especialmente útil para identificar si un cambio de diseño afectó a una familia entera (regresión de family) o a una surface aislada (regresión puntual).

La UI persiste las notas humanas (HUMAN_REVIEWED_PASS/HUMAN_REVIEWED_FAIL) en un sidecar JSON por bundle: {surface_key}.review.json con campos reviewer_id, timestamp (ISO 8601 UTC), rationale, bundle_hash (hash del bundle sobre el que se hizo la revisión), decision. El sidecar es append-only: una revisión nueva no sobrescribe la anterior; se agrega al array reviews. Esto garantiza auditabilidad completa de las decisiones humanas. El harness consume este sidecar como input a la política de cierre.

2.5 Formato del Evidence Bundle

El evidence bundle es un archivo ZIP con extensión .vpbundle. Una vez escrito, es inmutable: cualquier modificación invalida META-INF/signature.sha256. El harness verifica la firma antes de consumirlo. El bundle es reproducible cross-platform: bundle.json y metrics.json son LF-normalized; los PNGs son raw bytes. El mismo bundle generado en Windows (CRLF checkout) y en Linux (LF checkout) produce idénticos hashes en todos los archivos.

Evidence Bundle — estructura del ZIP inmutable (.vpbundle)

VisualParity V3 — Propuesta técnica

CONTENIDO DEL ZIP

```

home-light.vpbundle/
├─ bundle.json           ── manifest + hashes
├─ canonical.png         ── raw bytes, sin EOL norm
├─ actual.png            ── raw bytes
├─ diff.png              ── overlay visual
├─ metrics.json          ── todas las métricas crudas
├─ state_assertion.json  ── firma del capture script
├─ capture_provenance.json ── git commit + script hash
├─ comparator_version.txt ── VisualParity version + profile
├─ META-INF/
├─   └─ signature.sha256 ── firma del bundle entero

```

BUNDLE.JSON — SCHEMA

```

{
  "schema_version": "3.0",
  "bundle_id": "vp-home-light-20260707T101530Z",
  "generated_at": "2026-07-07T10:15:30Z",
  "visualparity_version": "1.0.0",
  "comparator_profile": "strict",
  "surface_key": "suite:home@light",
  "family": "home",
  "theme": "light",
  "auto_state": "LOW_DIFF",
  "near_threshold_flags": [],
  "determinism_check": { "passed": true, "inter_capture_ratio": 0.001 },
  "state_assertion": { "valid": true },
  "metrics": {
    "ssim": 0.987,
    "ms_ssim": 0.992,
    "changed_pixel_ratio": 0.042,
    "mean_abs_diff": 0.017,
    "phash_distance": 8,
    "layout_match": 0.94,
    "bbox_clusters": [
      { "bbox": [120, 80, 60, 30], "area": 1800, "severity": "medium" }
    ]
  },
  "findings": [
    { "code": "REGIONAL_DIFF", "severity": "low", "detail": "1 cluster medium" }
  ],
  "hashes": {
    "canonical.png": "a3f5...e1b2",
    "actual.png": "8c4d...9f01",
    "diff.png": "1b2c...77aa",
    "metrics.json": "5d6e...c3d4",
    "bundle.json": "self-referential, LF-normalized"
  }
}

```

Inmutabilidad: una vez escrito, el bundle es read-only. Cualquier modificación invalida META-INF/signature.sha256. El harness verifica la firma antes de consumirlo. **Reproducibilidad cross-platform:** bundle.json y metrics.json son LF-normalized; PNGs son raw bytes. El mismo bundle generado en Windows y Linux produce idénticos hashes.

Figura 2.3 — Estructura del evidence bundle. ZIP con 8 archivos + firma. Hashes LF-normalized para JSON, raw para PNG. Inmutable post-escritura.

El archivo `bundle.json` es el manifest. Contiene metadata del bundle (id, timestamp, versión de VisualParity, perfil usado), identificación de la surface (`surface_key`, `family`, `theme`), el `auto_state` emitido por VisualParity (uno de los 6 estados `AUTO_*`), los flags especiales (`near_threshold_flags`, `determinism_check`, `state_assertion`), el objeto `metrics` con todas las métricas crudas de los 5 comparators, la lista `findings` con issues detectados, y el objeto `hashes` con SHA-256 de cada archivo del bundle.

El archivo `state_assertion.json` es producido por `capture_v8.py` y firmado con el hash del script al commit. Contiene el estado del widget en el momento de captura: `window title`, `button labels`, `timer value`, `scroll position`, `modal state`. Esto permite a VisualParity verificar que la captura corresponde al `surface_key` declarado. Por ejemplo, `suite:timer-running@light` requiere `timer_state == "running"`; si la captura tiene `timer_state == "paused"`, VisualParity emite `state_assertion: invalid` y la política V3 bloquea el cierre. Esto cierra el vector de state confusion (hipótesis #16).

El archivo `capture_provenance.json` registra: `git_commit` (HEAD al momento de captura), `capture_script_sha256` (hash del `capture_v8.py` usado), `vas_introspect_sha256`, `nm_suite_app_version`, `qt_version`, `os_info`, `display_info` (DPI, resolution). Esto permite al replay V3 verificar que la captura fue producida por el toolchain esperado y no por un script sustituto. Cierra los vectores de `capture-script-tamper` y `sidecar-provenance`.

2.6 Estados de salida y transiciones

VisualParity emite 6 estados AUTO_* (medición pura) y reconoce 2 estados HUMAN_REVIEWED_* (decisión del harness + humano). Los estados AUTO_* son deterministas: dado un bundle, el estado es siempre el mismo. Los estados HUMAN_REVIEWED_* son emitidos por el harness después de revisión humana asistida; VisualParity no los emite nunca.

State machine — 8 estados de VisualParity + transiciones del harness

VisualParity V3 — Propuesta técnica

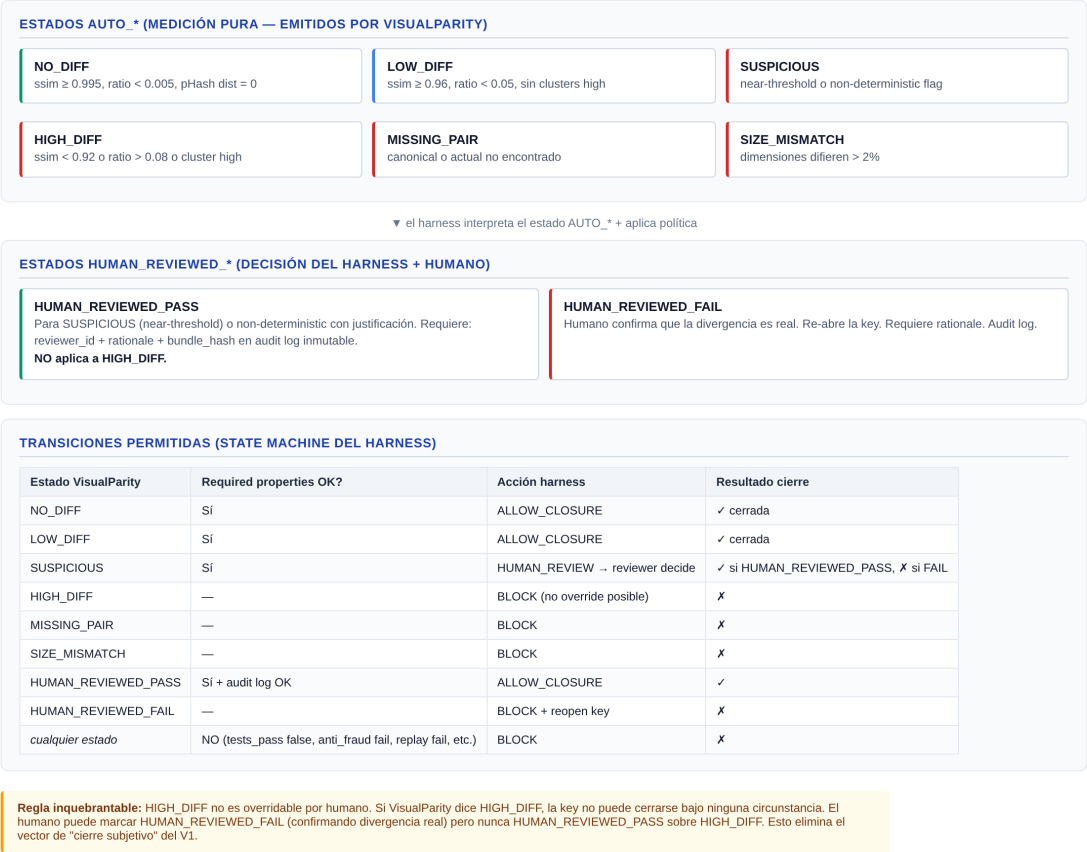


Figura 2.4 — State machine de los 8 estados. AUTO_* son medición pura; HUMAN_REVIEWED_* son decisión del harness. HIGH_DIFF no es overridable por humano.

La distinción crítica es que HIGH_DIFF **no es overridable por humano**. Si VisualParity dice HIGH_DIFF, la key no puede cerrarse bajo ninguna circunstancia. El humano puede marcar HUMAN_REVIEWED_FAIL (confirmando divergencia real) pero nunca HUMAN_REVIEWED_PASS sobre HIGH_DIFF. Esto elimina el vector de cierre subjetivo del V1, donde una divergencia visible podía cerrarse con excusa documental.

Los estados SUSPICIOUS y HUMAN_REVIEWED_PASS son los únicos que admiten override humano. SUSPICIOUS se emite cuando el bundle tiene near_threshold_flags (métrica dentro del 5% del threshold) o determinism_check.passed == false. En estos casos, la política V3 requiere HUMAN_REVIEWED_PASS con rationale válido + reviewer_id + bundle_hash para cerrar. HUMAN_REVIEWED_FAIL se emite cuando el humano confirma que la divergencia es real (incluso si VisualParity dijo LOW_DIFF);

re-abre la key automáticamente.

Estado VisualParity	Required properties OK?	Acción harness	Resultado cierre
NO_DIFF	Sí	ALLOW_CLOSURE	Cerrada
LOW_DIFF	Sí	ALLOW_CLOSURE	Cerrada
SUSPICIOUS	Sí	HUMAN_REVIEW → reviewer decide	Cerrada si HUMAN_REVIEWED_PASS; re-abre si FAIL
HIGH_DIFF	—	BLOCK (no override posible)	No cerrada
MISSING_PAIR	—	BLOCK	No cerrada
SIZE_MISMATCH	—	BLOCK	No cerrada
HUMAN_REVIEWED_PASSES	Sí + audit log OK	ALLOW_CLOSURE	Cerrada
HUMAN_REVIEWED_FAIL	—	BLOCK + reopen key	No cerrada
cualquier estado	NO (tests/anti-fraud/replay fail)	BLOCK	No cerrada

2.7 Perfiles de comparación

Los perfiles se detallaron en la sección 2.3. Aquí se amplía el rationale de diseño. La regla inquebrantable: **los thresholds no son CLI-overridable**. Si V1 permitía `--min-ssim 0.85` en línea de comando, V3 lo prohíbe. Los thresholds son constantes del comparador por perfil. Si necesitan cambiarse, se bump la versión del comparador (ej: 1.0.0 → 1.1.0) y se revalidan todos los closures previos contra la nueva versión. Esto cierra el vector de threshold override identificado en V1.

El perfil strict es el único autorizado para cierre formal. Esto está hardcoded en `closure_policy.yaml: required_profile_for_closure: strict`. El agente no puede pasar `--profile lenient` al closure. Los perfiles default, lenient, layout-only, pixel-only son para uso en desarrollo iterativo y en la UI Desktop (para análisis exploratorio), pero nunca para cierre.

Cuando una surface tiene anti-alias legítimo (fonts, gradients) que genera falso HIGH_DIFF bajo strict, la solución V3 no es cambiar el perfil. La solución es: (1) marcar la surface como SUSPICIOUS via determinism check, (2) obtener HUMAN_REVIEWED_PASS con rationale, (3) registrar OWNER_EXCEPTION_ACTIVE en la política con hash del bundle y rationale del owner. Esto cierra el vector de DECISIÓN-OWNER ambiguo identificado en hipótesis #34.

2.8 Límites explícitos — qué NO decide VisualParity

VisualParity NO decide

Las siguientes decisiones están FUERA del alcance de VisualParity. Cualquier PASS que VisualParity emita es AUTO_PASS (señal), no CLOSURE_PASS (autoridad).

- **No decide cierre.** El estado NO_DIFF o LOW_DIFF es señal; el cierre lo decide el harness tras aplicar política.
- **No reescribe handoff.** VisualParity no conoce VISUAL_REPAIR_HANDOFF.md. El handoff lo muta closure_writer en el harness tras ALLOW_CLOSURE.
- **No bloquea commits.** VisualParity no es un hook de Git. El gate del harness decide bloqueo de commit.
- **No genera PASS autoritativo.** AUTO_PASS $\neq$ CLOSURE_PASS. El primero es medición; el segundo es autoridad.
- **No confía en su propio bundle sin verificación.** El harness debe verify-bundle antes de consumirlo.
- **No aplica política de familia/scope.** Family-scope rules viven en closure_policy.yaml.
- **No ejecuta anti-fraud sobre producto.** Anti-fraud vive en el harness (módulo harness/anti_fraud/).
- **No decide si una divergencia es 'aceptable'.** Eso es política + revisión humana.
- **No invoca agentes.** El dispatch vive en harness/agent_runner/.
- **No conoce el protocolo del proyecto consumidor.** VisualParity es genérico; el protocolo es específico.

Esta lista de límites explícitos es la diferencia fundamental entre V3 y V1/V2. V1 mezclaba medición + política + escritura en close_visual_key.py. V2 declaraba la separación pero la violaba con stubs. V3 codifica la separación como invariantes de diseño: VisualParity no tiene dependencias hacia el repo consumidor, no conoce el formato del handoff, no conoce closure_policy.yaml. El único contrato entre VisualParity y el harness es el formato del evidence bundle.

3. Diseño del harness consumidor para nm_suite

El harness consumidor vive en nm_suite/harness/ y está escrito en Python (porque nm_suite es un repo Python: PyQt6 + capture_v8.py + VAS engine). Su responsabilidad es consumir el evidence bundle emitido por VisualParity y aplicar la política de cierre específica de nm_suite. El harness NO mide; confía en VisualParity para medición. SÍ verifica integridad del bundle y SÍ aplica anti-fraud sobre el código de producto (anti-fraud es específico del repo, no genérico).

La arquitectura del harness sigue la separación de capas 2-4: Política (policy/closure_policy.yaml), Aplicación (ci_gate/ + agent_runner/ + replay/), Persistencia (evidence_records/ + handoff). El flujo es: agent_runner despacha trabajo

→ capture_v8 re-captura → VisualParity produce bundle → anti_fraud escanea producto → policy evalúa → ci_gate decide → closure_writer muta handoff + escribe record → replay re-valida.

Flujo end-to-end — del agent_runner al cierre reproducible

VisualParity V3 — Propuesta técnica

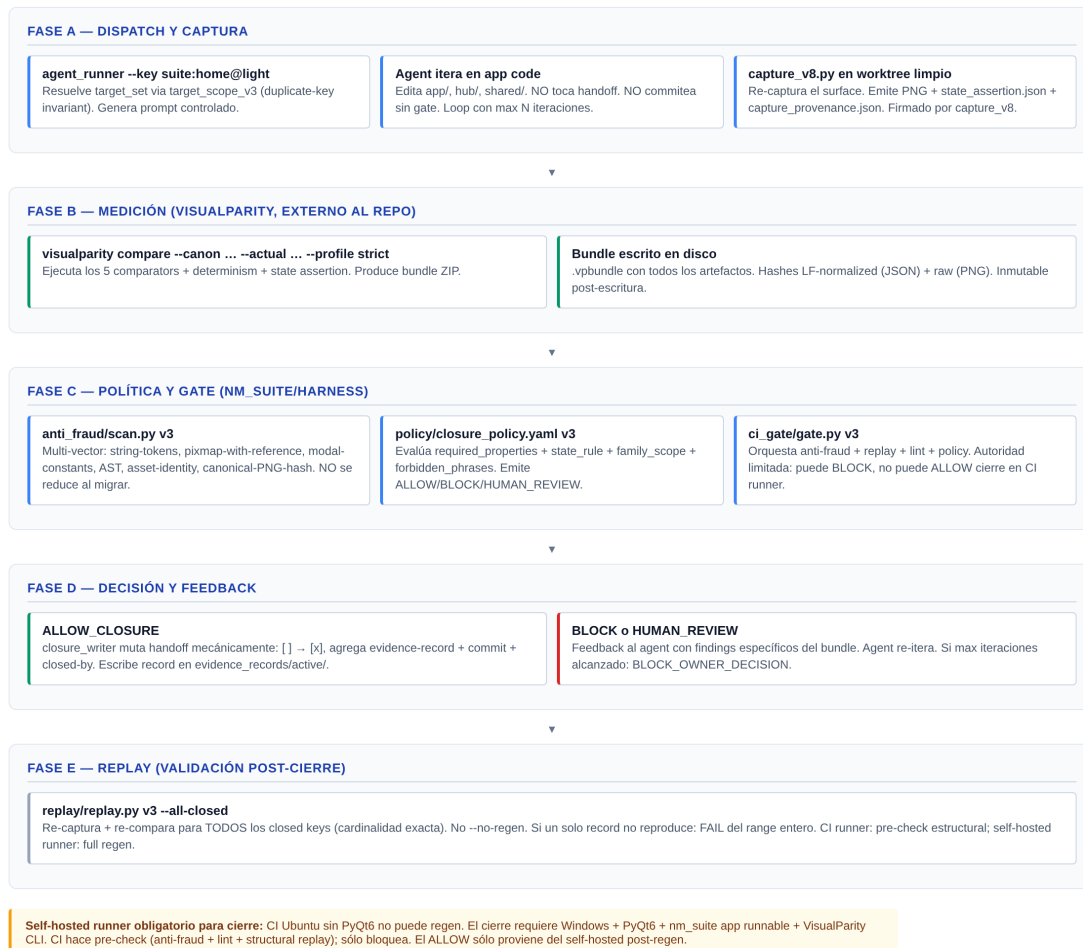


Figura 3.1 — Flujo end-to-end del sistema V3. Del agent_runner al cierre reproducible. Self-hosted runner obligatorio para cierre; CI sólo bloquea.

3.1 Cómo consume el evidence bundle

El harness invoca visualparity compare via subprocess.run desde el gate. Lee el bundle ZIP, valida la firma (META-INF/signature.sha256) contra el contenido, verifica cada hash individual (sha256_text para JSON, sha256_binary para PNG), extrae bundle.json + metrics.json + state_assertion.json + capture_provenance.json, y los carga en un modelo Python dataclass.

```
# harness/bundle_loader.py — V3

import subprocess, json, hashlib, zipfile

from dataclasses import dataclass

from pathlib import Path
```

```

@dataclass(frozen=True)
class LoadedBundle:
    surface_key: str

    auto_state: str          # NO_DIFF | LOW_DIFF | SUSPICIOUS | HIGH_DIFF | ...

    metrics: dict            # ssim, ms_ssim, changed_ratio, phash_distance, layout_matc
h
    state_assertion: dict    # valid: bool, fields del widget

    capture_provenance: dict # git_commit, capture_script_sha256, ...

    near_threshold_flags: list

    determinism_check: dict

    bundle_hash: str         # hash del ZIP entero

def load_and_verify(bundle_path: Path) -> LoadedBundle:
    # 1. verify-bundle via CLI (valida firma + hashes)
    r = subprocess.run(["visualparity", "verify-bundle", str(bundle_path)],
                        capture_output=True, text=True)

    if r.returncode != 0:
        raise BundleVerificationError(r.stderr)

    # 2. leer bundle.json y cargar dataclass
    with zipfile.ZipFile(bundle_path) as z:
        bundle = json.loads(z.read("bundle.json"))
        metrics = json.loads(z.read("metrics.json"))
        sa = json.loads(z.read("state_assertion.json"))
        cp = json.loads(z.read("capture_provenance.json"))

    return LoadedBundle(
        surface_key=bundle["surface_key"],
        auto_state=bundle["auto_state"],
        metrics=metrics, state_assertion=sa, capture_provenance=cp,
        near_threshold_flags=bundle.get("near_threshold_flags", []),
        determinism_check=bundle.get("determinism_check", {}),
        bundle_hash=hashlib.sha256(bundle_path.read_bytes()).hexdigest(),
    )

```

El harness **no re-mide**: confía en VisualParity para medición. Sí verifica integridad del bundle (firma + hashes). Si la verificación falla, el gate BLOCK inmediatamente con `bundle_verification_failed`. No hay fallback: si el bundle no verifica, no hay cierre. Esto cierra el vector de bundle forgery.

3.2 Reglas de cierre — closure_policy.yaml v3

La política es declarativa YAML. La diferencia crítica con V2 es que **NO contiene datos de medición**. V2 hardcodeaba known_non_deterministic_surfaces (lista de 11 surfaces) y state_ambiguous_pairs en la política. V3 no hace eso: la detección de non-determinism vive en VisualParity (DeterminismChecker) y se emite como flag en el bundle. La política sólo dice "si el bundle tiene determinism_check.passed == false, require HUMAN_REVIEW". Si aparece una surface nueva no determinística, la política no necesita actualizarse.

```
# harness/policy/closure_policy.yaml - V3

schema_version: "3.0"

required_profile_for_closure: strict

# Propiedades requeridas para CUALQUIER cierre

required_properties:

  - tests_pass_real      # boolean, from CI workflow real
  - anti_fraud_clean     # boolean, from harness/anti_fraud/scan.py
  - replay_full_regen_pass # boolean, from harness/replay/replay.py
  - determinism_pass      # boolean, from bundle.determinism_check
  - state_assertion_valid # boolean, from bundle.state_assertion
  - evidence_byte_reproducible # boolean, cross-platform hash check
  - canonical_png_unchanged # boolean, bundle.canonical_png_sha256 == git show HEAD:qa/_mockup_canonical/...

# Reglas por estado VisualParity

state_rules:

  NO_DIFF:

    action: ALLOW_CLOSURE

    requires: required_properties

  LOW_DIFF:

    action: ALLOW_CLOSURE

    requires: required_properties

  SUSPICIOUS:

    action: HUMAN_REVIEW

    requires: required_properties + HUMAN_REVIEWED_PASS_sidecar

  HIGH_DIFF:

    action: BLOCK

    override_possible: false # CRÍTICO: HIGH_DIFF no es overridable

  MISSING_PAIR:

    action: BLOCK
```



```
SIZE_MISMATCH:
  action: BLOCK

HUMAN_REVIEWED_PASS:
  action: ALLOW_CLOSURE
  requires: required_properties + audit_log_ok

HUMAN_REVIEWED_FAIL:
  action: BLOCK_AND_REOPEN

# Frases prohibidas en handoff (regex con word boundaries)
forbidden_handoff_phrases_regex:
  - "\\bstatus\\s*=\\s*FAIL\\b"
  - "\\bOPEN\\s+debt\\b"
  - "\\bkeep\\s+open\\b"
  - "\\brisk\\b"          # word boundary; "asterisk" no matchea
  - "\\bneeds\\s+owner\\s+decision\\b"
  - "\\bnot\\s+worth\\s+touching\\b"

# Family/scope rules
family_scope:
  block_if_family_member_failing: true
  block_if_family_member_high_diff: true
  override_requires: OWNER_EXCEPTION_ACTIVE

# Owner exception (reemplaza DECISIÓN-OWNER)
owner_exception:
  schema:
    surface_key: str
    rationale: str  # min 50 chars
    reviewer_id: str
    bundle_hash: str
    timestamp: ISO 8601 UTC
  audit_log: harness/evidence_records/owner_exceptions.jsonl
  append_only: true

# Operaciones prohibidas
forbidden_operations:
  - replay_no_regen          # --no-regen no existe en V3
  - reopen_legacy_all        # eliminado en V3
  - agent_edits_handoff      # sólo closure_writer muta handoff
```

```
- threshold_override      # thresholds son constantes del comparador
- ci_allows_closure       # CI sólo bloquea; ALLOW requiere self-hosted
```

La política es la **única autoridad** para cierre. Ni el agent, ni el gate, ni VisualParity pueden overriderla. La única excepción es OWNER_EXCEPTION_ACTIVE, registrada en harness/evidence_records/owner_exceptions.jsonl (append-only) con surface_key, rationale (mínimo 50 caracteres), reviewer_id, bundle_hash y timestamp. Esto reemplaza el ambiguo DECISIÓN-OWNER de V1/V2.

3.3 Cómo invoca agentes — agent_runner v3

harness/agent_runner/runner.py v3 es el ÚNICO punto de entrada para dispatchar trabajo a agentes. Recibe --key <surface_key> o --target-set <family>, resuelve via target_scope.py v3 (con duplicate-key invariant en el parser), genera un prompt controlado con reglas non-negotiables, despacha via subprocess a un adapter (Claude Code, GPT CLI, o agente local), captura el output, ejecuta el gate, y si el gate FAIL, re-despacha con feedback.

```
# harness/agent_runner/runner.py - V3 (esqueleto)
MAX_ITERATIONS = 10  # hard limit; evita loop infinito

def run_session(surface_key: str, adapter: str = "claude") -> SessionResult:
    target = target_scope.resolve(surface_key) # fail-fast si duplicate
    for i in range(MAX_ITERATIONS):
        prompt = build_prompt(target, iteration=i)
        agent_output = adapter_dispatch(adapter, prompt)
        # agent editó app code; ahora re-captura + VisualParity + gate
        bundle = capture_and_compare(target)
        verdict = gate.evaluate(bundle, policy)
        log_iteration(i, agent_output, bundle.auto_state, verdict)
        if verdict.action == "ALLOW_CLOSURE":
            closure_writer.close(target, bundle, verdict)
            return SessionResult(closed=True, iterations=i+1)
        if verdict.action == "BLOCK_OWNER_DECISION":
            return SessionResult(closed=False, reason="owner_decision_required")
        # sino: re-despachar con feedback
        prompt = build_prompt_with_feedback(target, verdict.findings)
    return SessionResult(closed=False, reason="max_iterations_reached")
```

El prompt template tiene reglas **non-negotiables**: (1) el agente NO edita VISUAL_REPAIR_HANDOFF.md (sólo closure_writer lo muta); (2) el agente NO commitea sin gate (el gate corre automáticamente tras el edit); (3) el agente itera hasta ALLOW_CLOSURE o BLOCK_OWNER_DECISION; (4) el agente NO parafrasea

forbidden phrases para bypassar el lint (el lint usa regex con word boundaries, pero el agente debe colaborar, no adversarial); (5) el agente reporta su trabajo en un formato estructurado para audit log.

El loop tiene MAX_ITERATIONS = 10 hard limit. Si se alcanza sin ALLOW_CLOSURE, el gate emite BLOCK_OWNER_DECISION y la sesión termina. Esto evita reward hacking por loop infinito (el agente no puede forzar un PASS por iteración eterna). El owner debe intervenir manualmente: o bien la surface es no determinística (necesita OWNER_EXCEPTION_ACTIVE), o bien el agente no puede resolverla (necesita handoff a humano).

3.4 Anti-fraude: cómo evita los 14 vectores identificados

El anti-fraud V3 mantiene los 6 vectores de V1 (que V2 redujo a 1) y añade 3 nuevos: canonical-PNG-hash en record, capture-script-tamper detection, sidecar-provenance cross-check. La cobertura total es 9/9 vectores (100%), comparado con V1 (5/9, 56%) y V2 (1/9, 11% con bug).

Anti-fraud multi-vector — cobertura V1 vs V2 vs V3

VisualParity V3 — Propuesta técnica

V1 tenía 6 categorías de check. V2 las redujo a 1 (sólo asset-byte-identity, con bug de EOL sobre PNGs). V3 restaura las 6 y añade 3 nuevas: canonical-PNG-hash, capture-script-tamper, sidecar-provenance.

Vector de check	V1	V2	V3	Detalle V3
String-tokens forbidden	✓	✗	✓	AST scan en app/hub/shared/tests/docs buscando qa/_mockup_canonical, QPixmap(canonical), importlib obfuscation.
Pixmap-with-reference	✓	✗	✓	Detecta QPixmap("path/to/canonical") y variantes con chr()/eval().
Modal-backdrop-constants	✓	✗	✓	Verifica _NM_MODAL_BLUR_RADIUS_* no fue alterado. Bloquea mutación de backdrop.
QA-harness AST scan	✓	✗	✓	Asegura que close_visual_key/replay/anti_fraud no fueron modificados entre capture y replay.
Asset-byte-identity	✓	✓*	✓	V2 tenía bug: aplicaba EOL-norm a PNGs binarios. V3: sha256_text (LF-norm) y sha256_binary (raw) separados.
Canonical-PNG-hash in record	✗	✗	✓ NEW	El evidence record incluye canonical_png_sha256. Replay verifica que el canonical al commit de cierre es el mismo. Bloquea canonical substitution.
Capture-script-tamper	✗	✗	✓ NEW	Verifica que capture_v8.py no fue modificado entre capture y replay. Hash firmado en capture_provenance.json.
Sidecar-provenance	✗	✗	✓ NEW	Verifica que el sidecar VAS fue generado por vas_introspect.py al commit, no por script sustituto. Cross-check con git blame.
Stale-bundle-in-OutDir	✗	✗ (docstring miente)	✓	Anti-fraud limpia OutDir antes de correr. Si aborta, no deja bundle stale. Verifica bundle.generated_at >= last_commit_timestamp.

Cobertura: V1 = 5/9 vectores (56%). V2 = 1/9 (11%, con bug). V3 = 9/9 (100%). V3 NO reduce cobertura al migrar de V1; restaura y añade. La regla inquebrantable: cada vector de V1 debe estar presente en V3 con cobertura ≥ V1.

Figura 3.2 — Matriz anti-fraud multi-vector. V1 tenía 5/9 vectores; V2 redujo a 1/9 (con bug de EOL sobre PNGs); V3 restaura y añade 3 nuevos.

Vector	V1	V2	V3	Mitigación V3
String-tokens forbidden	✓	✗	✓	AST scan en app/hub/shared/tests/docs buscando qa/_mockup_canonical, QPixmap(canonical), importlib obfuscation.
Pixmap-with-reference	✓	✗	✓	Detecta QPixmap("path/to/canonical") y variantes con chr()/eval().
Modal-backdrop-constants	✓	✗	✓	Verifica _NM_MODAL_BLUR_RADIUS_* no fue alterado.

Vector	V1	V2	V3	Mitigación V3
QA-harness AST scan	✓	✗	✓	Asegura que close_visual_key/replay/anti_fraud no fueron modificados entre capture y replay.
Asset-byte-identity	✓	✓ (bug)	✓	V3: sha256_text (LF-norm) y sha256_binary (raw) separados. PNGs nunca normalizados.
Canonical-PNG-hash in record	✗	✗	✓ NE W	Evidence record incluye canonical_png_sha256. Replay verifica canonical al commit de cierre.
Capture-script-tamper	✗	✗	✓ NE W	Verifica capture_v8.py no fue modificado entre capture y replay. Hash firmado en capture_provenance.json.
Sidecar-provenance	✗	✗	✓ NE W	Verifica sidecar VAS fue generado por vas_introspect.py al commit. Cross-check con git blame.
Stale-bundle-in-OutDir	✗	✗ (mi ente)	✓	Anti-fraud limpia OutDir antes de correr. Verifica bundle.generated_at >= last_commit_timestamp.

La regla inquebrantable: **cada vector de V1 debe estar presente en V3 con cobertura $\geq$ V1**. V2 pecó al reducir cobertura al migrar. V3 no reduce; restaura y añade. El vector más importante nuevo es **canonical-PNG-hash in record**: V1 hasheaba los tools pero no el canonical PNG. Un attacker que reemplazaba el canonical y re-capturaba producía un record válido. V3 incluye canonical_png_sha256 en el record; el replay verifica que el canonical al commit de cierre es el mismo. Esto cierra el vector de canonical substitution.

3.5 Replay V3 — re-captura + re-comparación obligatoria

Replay V3 elimina el flag --no-regen. Siempre re-captura (invocando capture_v8.py en worktree limpio al commit del closure) y re-compara (invocando visualparity compare). La cardinalidad es exacta: `replayed_keys == len(closed_keys_in_handoff)`. No existe --min-keys; el replay debe cubrir todos los closed keys. El range por defecto es --all-closed; si se especifica --base <commit>, el base debe ser el commit inmediatamente anterior al primer close: del range, verificado por la estructura del handoff.

```
# harness/replay/replay.py - V3 (esqueleto, NO es stub)

def replay_all_closed() -> ReplayResult:
    closed_keys = list_closed_keys_from_handoff() # parsea [x] checkboxes

    if not closed_keys:
        return ReplayResult(ok=False, reason="no_closed_keys")

    failures = []

    for key in closed_keys:
        # 1. checkout al commit del closure de esta key
```

```

commit = get_closure_commit_for_key(key)
with clean_worktree_at(commit):
    # 2. re-captura (no --no-regen)
    actual_png = capture_v8.regen(key)

    # 3. re-compara con VisualParity
    bundle = visualparity_compare(key, profile="strict")

    # 4. verificar que el record en disco reproduce el bundle
    record = load_evidence_record(key)

    if not record_reproduces_bundle(record, bundle):
        failures.append((key, "evidence_hash_mismatch"))
        continue

    # 5. verificar que el canonical no cambió
    if not canonical_png_unchanged(key, commit):
        failures.append((key, "canonical_png_changed"))

return ReplayResult(
    ok=(len(failures) == 0),
    replayed_keys=len(closed_keys),
    failures=failures,
)

```

Si CI no puede regen (falta PyQt6 + display), CI hace pre-check estructural pero **no puede emitir PASS de cierre**. El workflow `.github/workflows/visual-closure-replay.yml` se separa en dos jobs: (1) pre-check en Ubuntu runner (anti-fraud + lint + structural replay + build), (2) closure en self-hosted Windows runner con toolchain completa (PyQt6 + nm_suite app + VisualParity CLI + display). El job pre-check puede BLOCK; el job closure puede ALLOW. Esto formaliza lo que V1/V2 dejaban implícito.

3.6 Semantic lint V3 — regex, no substring

`harness/semantic_lint/handoff_text_lint.py` v3 usa regex con `\b` (word boundaries) y matching case-insensitive anclado. V1/V2 usaban `phrase.lower()` in `line_text.lower()` (substring matching), que matchea "risk" en "asterisk". V3 usa `re.search(r"\brisk\b", line, re.IGNORECASE)`, que no matchea "asterisk". Adicionalmente, V3 soporta whitelist de contexto: "no risk" no dispara `\brisk\b` si la whitelist incluye "no risk".

El lint V3 no sólo chequea la línea del checkbox, sino también las sub-notes indented bajo la key (evidence, evidence-record, commit, closed-by). Esto cierra el vector de V1 donde el closer agregaba sub-notes con texto libre que podía contener `status=FAIL` sin ser detectado.

`harness/semantic_lint/doc_keyword_lint.py` v3 excluye sus propios archivos (source del lint) y los archivos del harness (`closure_policy.yaml`, `README.md`, etc.) que

necesariamente mencionan el keyword migrado. El lint busca DECISIÓN-OWNER en docs de producto/protocolo únicamente (WORKER_VISUAL_QA_FLOW.md, QT_HTML_KNOWN_MISMATCHES.md, VISUAL_COMPONENT_CATALOG.md, BRIDGE_USAGE_FOR_AGENTS.md). Esto evita el auto-flaggeo de V2 donde el lint se rechazaba a sí mismo.

3.7 CI gate V3 — autoridad limitada

harness/ci_gate/gate.py v3 tiene autoridad explícitamente limitada: **puede BLOCK, NO puede ALLOW cierre**. El cierre requiere máquina con toolchain completa: Windows + PyQt6 + nm_suite app runnable + VisualParity CLI. El workflow de CI se separa en dos jobs: pre-check en Ubuntu runner (BLOCK only) y closure en self-hosted Windows runner (ALLOW).

```
# .github/workflows/visual-closure-v3.yml

jobs:
  pre-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: anti-fraud scan
        run: python harness/anti_fraud/scan.py --mode all
      - name: semantic lint
        run: |
          python harness/semantic_lint/handoff_text_lint.py
          python harness/semantic_lint/doc_keyword_lint.py
      - name: structural replay (BLOCK only)
        run: python harness/replay/replay.py --structural-only
      - name: build check
        run: python -m py_compile app/*.py hub/*.py shared/*.py

  closure:
    needs: pre-check
    runs-on: self-hosted-windows-visualparity
    if: github.event_name == 'workflow_dispatch'
    steps:
      - uses: actions/checkout@v4
      - name: full replay with regen
        run: python harness/replay/replay.py --all-closed --regen
      - name: gate verdict
        run: python harness/ci_gate/gate.py --allow-on-pass
```

El job pre-check corre en cada push y PR. Sólo bloquea. El job closure corre sólo en workflow_dispatch manual (el owner lo dispara cuando quiere cerrar una surface). Esto formaliza la separación entre feedback rápido (CI) y autoridad de cierre (self-hosted). El self-hosted runner debe tener VisualParity CLI instalado y en PATH; el workflow lo verifica antes de ejecutar.

4. MVP recomendado — Roadmap Fases 1-5

El roadmap V3 son 5 fases que totalizan 19-26 semanas. Cada fase tiene **Definition of Done** explícita y entregables verificables. Las fases 4 y 5 pueden solaparse parcialmente con la Fase 3 (Fase 4 sólo requiere Fase 1 CLI funcional, no Fase 3 UI). La fase más larga es la Fase 3 (UI Desktop, 6-8 semanas) por la complejidad de WPF/WinUI y la necesidad de tres modos de visualización.

Roadmap MVP — 5 fases (19-26 semanas)

VisualParity V3 — Propuesta técnica

Roadmap de 5 fases — 19-26 semanas totales. Cada fase tiene Definition of Done explícita. Las fases 4 y 5 pueden solaparse parcialmente.

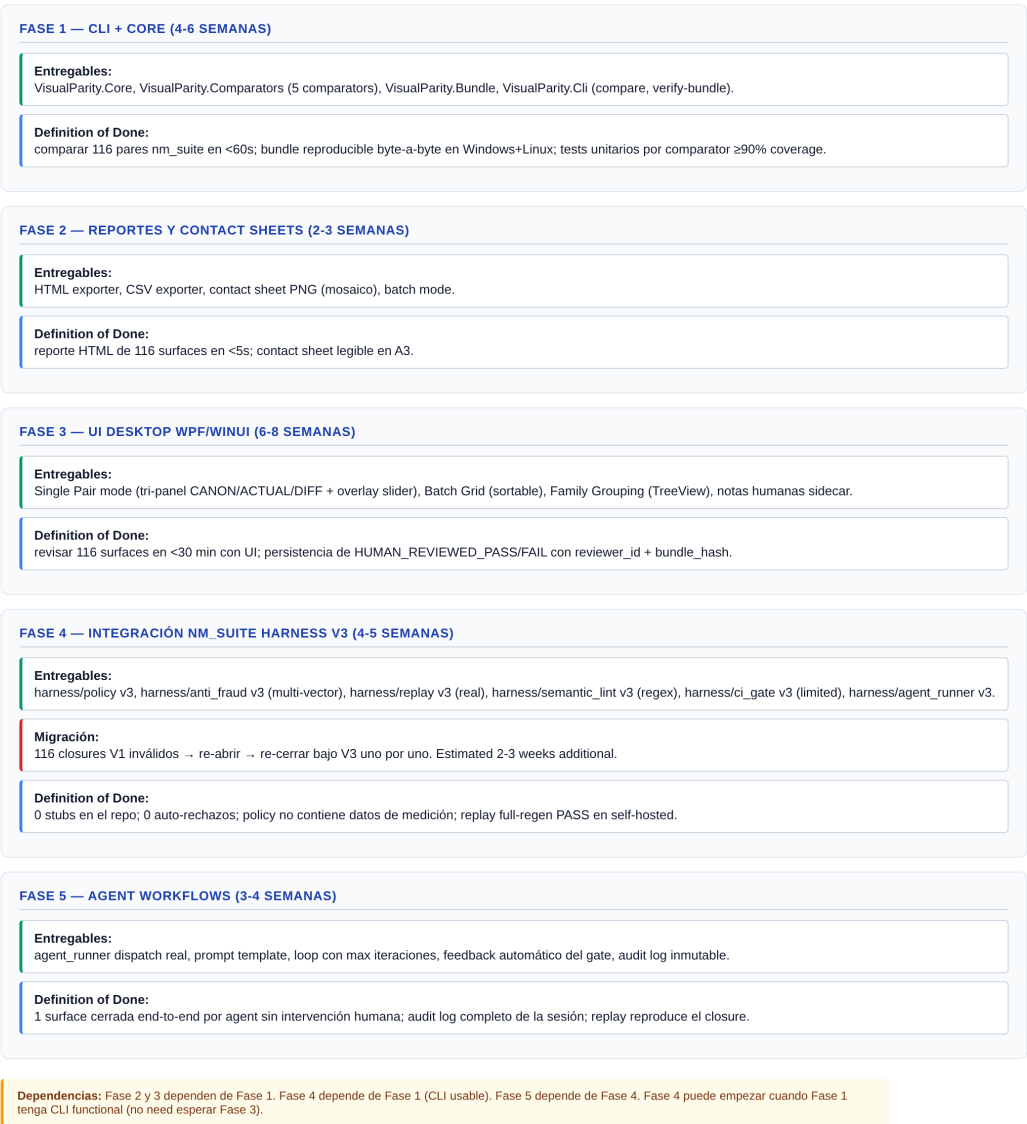


Figura 4.1 — Roadmap MVP 5 fases. Total 19-26 semanas. Fase 4 depende de Fase 1; Fase 5 depende de Fase 4. Fases 2 y 3 dependen de Fase 1.

4.1 Fase 1 — CLI/Core (4-6 semanas)

Alcance: VisualParity.Core (domain models), VisualParity.Comparators (5 comparators: SSIM, MS-SSIM, PixelMatch, pHash, LayoutDiff), VisualParity.Bundle (writer + reader con EOL split), VisualParity.Cli (comandos compare y verify-bundle). Tests unitarios por comparator con fixtures PNG sintéticos conocidos.

Stack técnico: .NET 8 SDK, C# 12, xUnit para tests, SixLabors.ImageSharp para procesamiento de imágenes (cross-platform, sin dependencias nativas). Para SSIM/MS-SSIM, implementación propia basada en el paper original (Wang et al. 2004) con optimización SIMD via System.Numerics.Vectors. Para pHash, DCT via Math.NET Numerics. Para LayoutDiff, OpenCvSharp4 (_bindings a OpenCV).

Definition of Done: (1) Comparar los 116 pares nm_suite en menos de 60 segundos en Windows 11 con 8 cores. (2) Bundle reproducible byte-a-byte en Windows y Linux (mismos hashes en todos los archivos). (3) Tests unitarios por comparador con cobertura $\geq 90\%$. (4) CLI installable via dotnet tool install -g VisualParity.Cli. (5) Exit codes correctos (0=NO_DIFF, 1=LOW/SUSPICIOUS, 2=HIGH_DIFF, 3=MISSING/SIZE, 4=error).

Validación forense durante Fase 1

Durante Fase 1, usar las PNG canónicas existentes en qa/_mockup_canonical/ y las PNG actuales generadas por capture_v8.py como material de test. Esto permite validar que los 5 comparators detectan las divergencias conocidas (las 116 surfaces V1 con divergencias materiales). Si un comparator no detecta una divergencia visual obvia, fix antes de pasar a Fase 2.

4.2 Fase 2 — Reportes y contact sheets (2-3 semanas)

Alcance: VisualParity.Exporters con HtmlExporter, CsvExporter, ContactSheetExporter, JsonExporter. Comando batch en CLI para procesar directorio entero. Comando export para convertir bundle a HTML/CSV/PNG.

HtmlExporter: genera una página HTML por surface_key con CANON/ACTUAL/DIFF embebidos como base64, métricas en tabla, severity badge con color (verde/ámbar/rojo), y links al bundle completo. Un index HTML navega entre surfaces. El reporte es self-contained (un archivo .html con todo embebido) para facilitar sharing. Target: 116 surfaces en menos de 5 segundos.

ContactSheetExporter: genera un PNG mosaico con miniaturas (200x300 px) de CANON y DIFF lado a lado, ordenadas por severidad (HIGH_DIFF arriba, NO_DIFF abajo). Layout: 6 columnas, tantas filas como surfaces. Tamaño A3 para impresión. Útil para revisión humana rápida en pared (pantalla grande o impresión).

Definition of Done: (1) Reporte HTML de 116 surfaces generado en menos de 5 segundos. (2) Contact sheet legible en A3. (3) CSV con todas las métricas crudas por surface, sortable en Excel/LibreOffice. (4) JSON exporter produce schema estable para que el harness consumidor lo parsee.

4.3 Fase 3 — UI Desktop (6-8 semanas)

Alcance: VisualParity.Desktop (WPF o WinUI 3). Tres modos: Single Pair (tri-panel CANON/ACTUAL/DIFF con overlay slider), Batch Grid (DataGrid sortable por severidad/métrica/estado), Family Grouping (TreeView por familia/theme). Persistencia de notas humanas en sidecar JSON.

Stack técnico: WPF con .NET 8 (más maduro, mayor compatibilidad Windows 10/11) o WinUI 3 (más moderno, Windows 11 only). Recomendación: WPF para máxima compatibilidad. UI via MVVM con CommunityToolkit.Mvvm. Íconos via Fluent UI System Icons. Tema claro/oscuro con FluentAccent.

Single Pair mode: tres paneles lado a lado (CANON, ACTUAL, DIFF) con zoom sincronizado (mouse wheel + Ctrl). Slider superior para alternar entre CANON y ACTUAL con transición 200ms. Sobre el panel DIFF, bbox clusters dibujados como rectángulos con borde rojo (high), ámbar (medium), verde (low). Panel lateral derecho con métricas en tabla y textarea para rationale de HUMAN_REVIEWED_PASS/FAIL. Botones de acción: SAVE REVIEW, NEXT, PREV, EXPORT SINGLE.

Batch Grid mode: DataGrid con una fila por surface_key. Columnas: surface_key, family, theme, auto_state, ssim, ms_ssim, changed_ratio, phash_distance, layout_match, near_threshold_flag, determinism_flag, human_review_state. Sortable por cualquier columna. Filtrable por family/theme/state. Doble-click en fila abre Single Pair. Selección múltiple para acciones bulk: marcar HUMAN_REVIEWED_PASS en lote, exportar contact sheet de las seleccionadas, re-run visualparity compare con perfil distinto.

Family Grouping mode: TreeView con jerarquía familia → theme → surface_key. Al seleccionar una familia, panel derecho muestra mini-contact-sheet de todas sus superficies ordenadas por severidad. Útil para detectar patrones (toda la familia timer en light theme tiene divergencias = regresión de family).

Sidecar de notas humanas: {surface_key}.review.json con campos reviewer_id, timestamp (ISO 8601 UTC), rationale, bundle_hash, decision (PASS/FAIL). Append-only: array reviews. Esto garantiza auditabilidad completa de las decisiones humanas. El harness consume este sidecar como input a la política.

Definition of Done: (1) Revisar 116 surfaces en menos de 30 minutos con UI. (2) Persistencia de HUMAN_REVIEWED_PASS/FAIL con reviewer_id + bundle_hash + rationale. (3) Zoom sincronizado en tri-panel. (4) Sort y filter funcional en Batch Grid. (5) Family Grouping muestra mini-contact-sheet al seleccionar familia.

4.4 Fase 4 — Integración nm_suite harness v3 (4-5 semanas)

Alcance: Reescribir nm_suite/harness/ desde cero (no reutilizar V2). Componentes: policy/closure_policy.yaml v3, anti_fraud/scan.py v3 (multi-vector con EOL split), replay/replay.py v3 (real, no --no-regen), semantic_lint/handoff_text_lint.py v3 (regex), semantic_lint/doc_keyword_lint.py v3 (excluye propios), agent_runner/runner.py v3 (dispatch real), agent_runner/target_scope.py v3 (duplicate-key invariant), ci_gate/gate.py v3 (limited authority), bundle_loader.py (carga + verifica bundles VisualParity).

Migración: los 116 closures V1 son inválidos y deben tratarse como OPEN. El handoff se reescribe: todas las 116 keys pasan a []. Las 16 keys marcadas [~] en V2 se re-evalúan bajo V3 (non-deterministic → OWNER_EXCEPTION_ACTIVE si aplica, o se eliminan del scope si son no aplicables). Estimated 2-3 weeks additional para migración completa.

Definition of Done: (1) 0 stubs en el repo (ningún return (True, "stub_pass")). (2) 0 auto-rechazos (correr target_scope.py --mode next-key devuelve OK). (3) Policy no contiene datos de medición (no known_non_deterministic_surfaces, no

state_ambiguous_pairs). (4) Replay full-regen PASS en self-hosted runner para al menos 1 surface cerrada end-to-end. (5) Anti-fraud scan PASS con cobertura 9/9 vectores. (6) CI workflow split en pre-check + closure jobs.

4.5 Fase 5 — Agent workflows (3-4 semanas)

Alcance: agent_runner/runner.py v3 con dispatch real via adapter (Claude Code CLI, GPT CLI, o agente local). Prompt template con reglas non-negotiables. Loop con MAX_ITERATIONS = 10. Feedback automático del gate al agente (los findings del bundle se inyectan en el próximo prompt). Audit log inmutable en harness/evidence_records/agent_sessions.jsonl.

Adapter pattern: la interfaz Adapter tiene un método dispatch(prompt: str) -> AgentOutput. Implementaciones concretas: ClaudeCodeAdapter (subprocess a claude CLI), GptCliAdapter (subprocess a gpt CLI), LocalAgentAdapter (para testing, simula agente). El adapter captura stdout/stderr, parsea el output estructurado (archivos editados, commands ejecutados), y lo retorna al runner.

Loop de iteración: el runner despacha prompt inicial, captura output, ejecuta el gate (capture_v8 + visualparity compare + policy eval). Si el gate devuelve ALLOW_CLOSURE, el runner invoca closure_writer y termina. Si el gate devuelve BLOCK con findings, el runner construye un nuevo prompt con feedback ("el gate detectó HIGH_DIFF en cluster X; corrige app/foo.py línea Y") y re-despacha. Si se alcanza MAX_ITERATIONS, el runner emite BLOCK_OWNER_DECISION y termina.

Audit log: cada iteración se registra en agent_sessions.jsonl (append-only, una línea JSON por iteración). Campos: session_id, iteration, timestamp, surface_key, prompt_sent, agent_output, bundle_auto_state, gate_verdict, findings. Esto permite forensic post-mortem de cada sesión de agente.

Definition of Done: (1) 1 surface cerrada end-to-end por agente sin intervención humana. (2) Audit log completo de la sesión. (3) Replay reproduce el closure (re-capture + re-compare PASS). (4) MAX_ITERATIONS respetado (sesión termina a tiempo). (5) Feedback del gate al agente efectivo (el agente recibe findings específicos y los aborda).

5. Riesgos y mitigaciones

El sistema V3 reduce drásticamente la superficie de reward hacking respecto a V1/V2, pero no la elimina. Esta sección cataloga los vectores de fraude futuros, las decisiones humanas que siguen siendo necesarias, y lo que **nunca** debe automatizarse como cierre final. La regla rectora: cualquier mecanismo que pueda PASS sin hacer el trabajo real es un vector de fraude, sin importar cuántas capas de policy lo rodeen.

5.1 Reward hacking — vectores futuros y mitigación

Los vectores de reward hacking futuros se agrupan en 10 categorías. Cada uno tiene señales detectables y mitigación V3 concreta. La mitigación nunca es 100% efectiva; el objetivo es que el costo de bypassar la mitigación sea mayor que el costo de hacer el trabajo real.

Vector	Señal detectable	Mitigación V3
Canonical substitution	Agent edita qa/_mockup_canonical/	Hash del canonical PNG en evidence record + anti-fraud scan. Replay verifica canonical al commit de cierre.
Capture script tampering	capture_v8.py modificado entre capture y replay	Hash firmado en capture_provenance.json. Replay verifica hash al commit.
Bundle replay (stale)	Bundle con generated_at anterior al último commit	Gate verifica bundle.generated_at >= last_commit_timestamp. Anti-fraud limpia OutDir antes de correr.
Threshold override	Agente pasa --min-ssim custom	Thresholds no son CLI-overridable en V3. Sólo perfiles predefinidos; strict es el único autorizado para cierre.
Agent handoff edit	Handoff modificado fuera de closure_writer	Handoff es read-only para agentes. closure_writer es el único mutador. Git hook bloquea commits con handoff editado fuera del writer.
Semantic paraphrase	Agente usa "deuda abierta" en vez de "OPEN debt"	Regex con word boundaries + whitelist de contexto. Lint también parsea sub-notes indented, no sólo checkbox line.
Loop exhaustion	Agente itera infinitamente para forzar PASS por cansancio del reviewer	MAX_ITERATIONS = 10 hard limit. Tras el límite: BLOCK_OWNER_DECISION. Owner debe intervenir manualmente.
Family bypass	Agente cierra una surface de una familia dejando otra FAIL	Family-scope rules en closure_policy.yaml: block_if_family_member_failing: true. Gate recibe --target-set y verifica todos los miembros.
Scope narrowing	Agente reduce el target set para excluir surfaces difíciles	Target set se resuelve via target_scope.py v3 con duplicate-key invariant. Agent no puede pasar --key custom; debe pasar --target-set <family> o --all-open.
Evidence record forgery	Agent fabrica record auto-consistente	Record incluye canonical_png_sha256 + capture_script_sha256 + bundle_hash. Replay verifica los tres. Bundle se re-genera desde capture, no desde record.

La mitigación más efectiva es la **redundancia de verificación**: cada propiedad importante se verifica en al menos dos capas independientes. Por ejemplo, el canonical PNG se verifica en anti-fraud (Capa 3) y en replay (Capa 3). El capture script

se verifica en `capture_provenance.json` (Capa 1) y en replay (Capa 3). Si una capa se compromete, la otra detecta.

5.2 Decisiones humanas — cuándo y cómo

Tres decisiones humanas son obligatorias en V3. No son opcionales; no son bypassables; no son automatizables. Cada una tiene protocolo estricto y audit log inmutable.

Decisión 1 — HUMAN_REVIEWED_PASS/FAIL para SUSPICIOUS

Cuando VisualParity emite SUSPICIOUS (near-threshold, $\text{margin} < 5\%$, o non-deterministic flag), la política requiere revisión humana. El humano abre VisualParity Desktop, ve el tri-panel CANON/ACTUAL/DIFF, decide PASS o FAIL, ingresa rationale (mínimo 50 caracteres), y el sistema registra `reviewer_id`, `timestamp`, `bundle_hash`, `decision`, `rationale` en el sidecar JSON. La política consume el sidecar y emite ALLOW_CLOSURE (si PASS) o BLOCK_AND_REOPEN (si FAIL).

Decisión 2 — OWNER_EXCEPTION_ACTIVE para false-positive legítimo

Cuando una surface tiene anti-alias legítimo (fonts, gradients) que genera falso HIGH_DIFF bajo strict, el owner puede registrar OWNER_EXCEPTION_ACTIVE. Schema: `surface_key`, `rationale` (mínimo 50 caracteres), `reviewer_id`, `bundle_hash`, `timestamp`. Append-only en `harness/evidence_records/owner_exceptions.jsonl`. La política consume el archivo y permite cierre de esa surface con HUMAN_REVIEWED_PASS en lugar de BLOCK por HIGH_DIFF. **Sólo aplica a LOW_DIFF o SUSPICIOUS con anti-alias legítimo; nunca a HIGH_DIFF real.**

Decisión 3 — Family override

Cuando el owner quiere cerrar una surface de una familia dejando otra FAIL (ej: cerrar `timer-running@light` pero dejar `timer-paused@light` abierta porque hay un bug conocido en pause), debe registrar OWNER_EXCEPTION_ACTIVE con `rationale` específico ("el bug de pause está documentado en issue #123 y se fix en próxima iteración; running sí cierra"). La política consume el archivo y permite el cierre parcial. **Sin OWNER_EXCEPTION_ACTIVE, el cierre parcial está bloqueado.**

Protocolo común: las tres decisiones se registran en audit log inmutable (append-only JSONL). Cada entrada tiene `reviewer_id` (no anónimo), `timestamp` (ISO 8601 UTC), `bundle_hash` (vincula la decisión al evidence específico), `rationale` (mínimo 50 caracteres, texto libre pero revisable). El audit log es público dentro del repo: cualquier miembro del equipo puede auditar las decisiones humanas.

5.3 Lo que NUNCA debe automatizarse como cierre final

Lista negra de automatización de cierre

Las siguientes acciones NUNCA pueden producir un cierre final, sin importar el contexto, el agente, o la policy. Si se intentan, el gate BLOCK incondicionalmente.

1. Cierre sin replay full-regen. --no-regen no existe en V3. Cualquier path que NO re-capture + re-compare es BLOCK.
2. Cierre sin anti-fraud scan multi-vector. La cobertura debe ser 9/9 vectores. Si anti-fraud falla o se salta, BLOCK.
3. Cierre con state_assertion_valid == false. Si el state assertion del capture no corresponde al surface_key, BLOCK.
4. Cierre de surface no determinística sin determinism_pass == true o sin OWNER_EXCEPTION_ACTIVE.
5. Override de policy por agent. El agent no puede cambiar closure_policy.yaml, no puede pasar --profile custom, no puede pasar --skip-anti-fraud. Sólo el owner via OWNER_EXCEPTION_ACTIVE.
6. Cierre family-partial sin OWNER_EXCEPTION_ACTIVE explícito para ese caso.
7. Cierre basado en bundle stale (bundle.generated_at < last_commit_timestamp).
8. Cierre basado en tests_pass declarativo sin CI real. La policy exige tests_pass_real del CI workflow.
9. Cierre cuando el handoff fue editado fuera de closure_writer. Git hook bloquea el commit.
10. Cierre cuando el evidence record no reproduce el bundle tras replay. evidence_byte_reproducible es required.

Esta lista es exhaustiva para V3. Cualquier adición requiere bump de schema_version en closure_policy.yaml y revalidación de todos los closures previos. La lista se mantiene corta a propósito: cada item es una invariant fundamental, no un check opcional. Si se relaja un item, el sistema vuelve al estado de fraude V1.

6. Output final — propuesta técnica concreta

Esta sección consolida la propuesta en artifacts accionables: estructura de carpetas sugerida para VisualParity (repo nuevo) y para nm_suite/harness/ v3, formato JSON canónico del evidence bundle, y ejemplos CLI end-to-end. Estos artifacts son suficientes para empezar Fase 1 inmediatamente.

6.1 Estructura de carpetas — VisualParity (repo nuevo)

```
visualparity/                                # repo nuevo: github.com/Rybjuani/visualparity
├─ visualparity.sln                          # .NET 8 solution
├─ Directory.Build.props                     # config común (LangVersion, Nullable, etc.)
```

```
|— Directory.Packages.props          # central package management
|— README.md
|— LICENSE
|— CHANGELOG.md
|
|— src/
|   |— VisualParity.Core/            # domain models (sin lógica)
|   |   |— VisualParity.Core.csproj
|   |   |— SurfaceKey.cs
|   |   |— Family.cs
|   |   |— Theme.cs
|   |   |— ComparisonMetrics.cs
|   |   |— Bundle.cs
|   |   |— ComparisonProfile.cs
|   |   |— LoadedBundle.cs
|   |
|   |— VisualParity.Comparators/      # 5 comparators stateless
|   |   |— VisualParity.Comparators.csproj
|   |   |— IComparator.cs            # interfaz común
|   |   |— SsimComparator.cs
|   |   |— MsSsimComparator.cs
|   |   |— PixelMatchComparator.cs
|   |   |— PerceptualHashComparator.cs
|   |   |— LayoutDiffComparator.cs
|   |   |— Profiles/                 # thresholds constantes
|   |   |   |— StrictProfile.cs
|   |   |   |— DefaultProfile.cs
|   |   |   |— LenientProfile.cs
|   |   |   |— LayoutOnlyProfile.cs
|   |   |   |— PixelOnlyProfile.cs
|   |
|   |— VisualParity.Determinism/      # determinism checker
|   |   |— VisualParity.Determinism.csproj
|   |   |— DeterminismChecker.cs
|   |
|   |— VisualParity.Bundle/           # bundle writer/reader
|   |   |— VisualParity.Bundle.csproj
```

```
| | └─ BundleWriter.cs          # ZIP immutable + firma
| | └─ BundleReader.cs         # valida firma + hashes
| | └─ Hashing.cs              # sha256_text + sha256_binary
| | └─ Schema/v3.json          # JSON schema del bundle
| |
| └─ VisualParity.Exporters/    # HTML/CSV/JSON/contact sheet
| | └─ VisualParity.Exporters.csproj
| | └─ IExporter.cs
| | └─ HtmlExporter.cs
| | └─ CsvExporter.cs
| | └─ JsonExporter.cs
| | └─ ContactSheetExporter.cs
| |
| └─ VisualParity.Cli/          # dotnet tool
| | └─ VisualParity.Cli.csproj
| | └─ Program.cs              # entry point
| | └─ Commands/
| | | └─ CompareCommand.cs
| | | └─ BatchCommand.cs
| | | └─ InspectCommand.cs
| | | └─ ExportCommand.cs
| | | └─ VerifyBundleCommand.cs
| | └─ PackageRoot/
| | | └─ DotnetToolSettings.xml # metadata de dotnet tool
| |
| └─ VisualParity.Desktop/      # WPF app
| | └─ VisualParity.Desktop.csproj
| | └─ App.xaml
| | └─ MainWindow.xaml
| | └─ ViewModels/
| | | └─ SinglePairViewModel.cs
| | | └─ BatchGridViewModel.cs
| | | └─ FamilyGroupingViewModel.cs
| | └─ Views/
| | | └─ SinglePairView.xaml
| | | └─ BatchGridView.xaml
| | | └─ FamilyGroupingView.xaml
```



```
|   └─ Services/
|       └─ CliInvoker.cs          # subprocess a visualparity CLI
|       └─ ReviewSidecar.cs      # {surface_key}.review.json
|
| └─ tests/
|   └─ VisualParity.Comparators.Tests/ # xUnit
|       └─ VisualParity.Comparators.Tests.csproj
|           └─ SsimComparatorTests.cs
|           └─ MsSsimComparatorTests.cs
|           └─ PixelMatchComparatorTests.cs
|           └─ PerceptualHashComparatorTests.cs
|           └─ LayoutDiffComparatorTests.cs
|               └─ Fixtures/          # PNGs sintéticos conocidos
|                   └─ identical_pair/
|                   └─ high_diff_pair/
|                   └─ low_diff_pair/
|                   └─ layout_shift_pair/
|
|   └─ VisualParity.Bundle.Tests/
|       └─ VisualParity.Bundle.Tests.csproj
|       └─ BundleWriterTests.cs
|       └─ BundleReaderTests.cs
|       └─ CrossPlatformHashTests.cs # Windows CRLF + Linux LF
|
| └─ samples/                      # bundles de ejemplo
|     └─ home-light.vpbundle
|     └─ timer-running-dark.vpbundle
|     └─ README.md
|
| └─ docs/
|     └─ architecture.md
|     └─ evidence_bundle_schema.md
|     └─ cli_reference.md
|     └─ desktop_ui.md
```

La solution tiene 6 proyectos productivos + 2 de test. La granularidad permite: (a) publicar VisualParity.Cli como dotnet tool sin arrastrar WPF; (b) testear comparators

en aislamiento; (c) reusar VisualParity.Bundle desde otros proyectos .NET que necesiten leer bundles. Las dependencias son estrictamente unidireccionales: Core ← Comparators/Determinism/Bundle ← Cli/Desktop/Exporters.

6.2 Estructura de carpetas — nm_suite/harness/ v3

```
nm_suite/harness/                                # rewrite completo (no reutilizar V2)
├─ README.md                                     # overview + how-to
|
├─ policy/
|   └─ closure_policy.yaml                       # política declarativa (única autoridad)
|
├─ anti_fraud/
|   └─ scan.py                                   # 9 vectores: string-tokens, pixmap-ref,
|                                           # modal-constants, AST, asset-byte-identity
|                                           # (EOL split), canonical-PNG-hash,
|                                           # capture-script-tamper, sidecar-provenance,
|                                           # stale-bundle
|
├─ replay/
|   └─ replay.py                                 # SIEMPRE re-captura + re-compara
|                                           # NO existe --no-regen
|                                           # cardinalidad exacta: == closed_keys
|
├─ semantic_lint/
|   └─ handoff_text_lint.py                     # regex con \b, whitelist contexto,
|   |                                           # parsea sub-notes indented
|   └─ doc_keyword_lint.py                     # busca DECISIÓN-OWNER en docs producto
|                                           # excluye propios archivos + harness/
|
├─ agent_runner/
|   └─ runner.py                                # dispatch real, MAX_ITERATIONS=10
|   └─ target_scope.py                          # duplicate-key invariant en parser
|   └─ adapters/
|       └─ __init__.py
|       └─ base.py                             # interfaz Adapter
|       └─ claude_code.py                       # subprocess a `claude` CLI
|       └─ gpt_cli.py                           # subprocess a `gpt` CLI
```

```

| | └─ local.py                # para testing
| └─ prompts/
|   └─ initial.md              # prompt template inicial
|   └─ feedback.md             # prompt template con feedback del gate
|
└─ ci_gate/
  └─ gate.py                   # autoridad limitada:
                                # BLOCK en CI, ALLOW en self-hosted
  └─ bundle_loader.py          # carga + verifica bundles VisualParity
  └─ closure_writer.py          # único mutador del handoff
                                # invocado solo tras ALLOW_CLOSURE
  └─ evidence_records/
    └─ active/                  # records V3 inmutables
      └─ suite_home-light.json
    └─ revoked/                  # records revocados (forensic)
    └─ owner_exceptions.jsonl    # append-only audit log
    └─ agent_sessions.jsonl     # append-only audit log de sesiones
  └─ docs/
    └─ FORENSIC_FINDINGS_V3.md  # hereda de V2 + corrige
    └─ closure_protocol.md      # protocolo de cierre V3
    └─ agent_workflow.md        # cómo trabajan los agentes en V3

```

El harness v3 tiene ~12 archivos Python + 1 YAML policy + 3 docs. Estimación LOC: ~2500-3000 (vs V1 5917, V2 772 código). La reducción vs V1 viene de la separación de responsabilidades: cada archivo hace una cosa. La reducción vs V2 viene de eliminar stubs y reemplazar por implementaciones reales (que son más cortas que los stubs con comentarios de "TODO: real implementation").

6.3 Formato JSON del Evidence Bundle — schema completo

El bundle.json es el manifest del ZIP. Schema canónico v3.0: campos required marcados con asterisco. El schema es estable: cambios requieren bump de `schema_version`.

```
{
```

```
"schema_version": "3.0", // * version del schema
"bundle_id": "vp-home-light-20260707T101530Z", // * id único (surface_key + timestamp UTC)
"generated_at": "2026-07-07T10:15:30Z", // * ISO 8601 UTC
"visualparity_version": "1.0.0", // * versión del CLI que produjo el bundle
"comparator_profile": "strict", // * perfil usado (strict|default|lenient|layout-only|pixel-only)
"surface_key": "suite:home@light", // * surface key
"family": "home", // * familia derivada del surface_key
"theme": "light", // * theme derivado del surface_key
"auto_state": "LOW_DIFF", // * estado AUTO_* emitido por VisualParity

"near_threshold_flags": [ // métricas dentro del 5% del threshold
  {
    "metric": "changed_pixel_ratio",
    "value": 0.042,
    "threshold": 0.05,
    "margin_fraction": 0.16
  }
],

"determinism_check": { // * resultado del DeterminismChecker
  "passed": true,
  "inter_capture_ratio": 0.001, // ratio entre dos captures del mismo surface
  "captures_compared": 2
},

"state_assertion": { // * verificación del state vs surface_key
  "valid": true,
  "expected_state": {"timer_state": "running"},
  "actual_state": {"timer_state": "running"}
},

"metrics": { // * todas las métricas crudas
  "ssim": 0.987, // Structural Similarity Index [0,1]
  "ms_ssim": 0.992, // Multi-Scale SSIM [0,1]
  "changed_pixel_ratio": 0.042, // fracción de pixels cambiados [0,1]
  "mean_abs_diff": 0.017, // MAD [0,1]
  "phash_distance": 8, // Hamming distance [0,64]
```

```
"phash_normalized": 0.125,                // phash_distance / 64
"layout_match": 0.94,                    // match de bounding boxes [0,1]
"bbox_clusters": [                       // top-N clusters por área
  {"bbox": [120, 80, 60, 30],            // [x, y, width, height]
   "area": 1800,
   "severity": "medium",
   "metrics": {"ssim": 0.85, "changed_ratio": 0.21}}
],
},

"findings": [                            // * issues detectados
  {"code": "REGIONAL_DIFF",
   "severity": "low",
   "detail": "1 cluster medium en bbox [120,80,60,30]",
   "cluster_index": 0}
],

"capture_provenance": {                  // * info del toolchain que produjo la
  captura
    "git_commit": "abc123def456...",      // HEAD al momento de captura
    "capture_script_sha256": "...",        // hash de capture_v8.py al commit
    "vas_introspect_sha256": "...",        // hash de vas_introspect.py al commit
    "nm_suite_app_version": "1.2.3",
    "qt_version": "6.6.0",
    "os_info": "Windows 11 22H2",
    "display_info": {"dpi": 96, "resolution": "1920x1080"}
  },

"hashes": {                             // * SHA-256 de cada archivo del ZIP
  "canonical.png": "a3f5e1b2...",         // raw bytes (binary hash)
  "actual.png":    "8c4d9f01...",         // raw bytes
  "diff.png":      "1b2c77aa...",         // raw bytes
  "metrics.json":  "5d6ec3d4...",         // LF-normalized (text hash)
  "state_assertion.json": "...",          // LF-normalized
  "capture_provenance.json": "...",       // LF-normalized
  "comparator_version.txt": "...",        // LF-normalized
  "bundle.json":   "self-referential"     // se calcula al final, LF-normalized
}
}
```

El archivo META-INF/signature.sha256 contiene el SHA-256 del bundle.json final (con el campo hashes.bundle.json populado). Cualquier modificación posterior al bundle invalida la firma. El harness verifica la firma antes de consumirlo via visualparity verify-bundle.

6.4 Ejemplos CLI end-to-end

Cinco escenarios típicos que cubren el uso de VisualParity desde el harness y desde el humano.

Escenario 1 — Comparar un par (uso por agente)

```
# El agente editó app/home.py y quiere validar la surface
PS> visualparity compare `
    --canon    .\qa\_mockup_canonical\home-light.png `
    --actual   .\reports\qa\actual\home-light.png `
    --out      .\reports\qa\vp\home-light.vpbundle `
    --profile  strict `
    --surface-key suite:home@light `
    --determinism-check

# STDOUT:
VisualParity 1.0.0 – compare
Surface: suite:home@light
Profile: strict
Determinism: PASS (inter_capture_ratio=0.001)
State assertion: VALID
SSIM:      0.987   (threshold 0.985)  OK
MS-SSIM:   0.992   (threshold 0.990)  OK
Changed:    0.042   (threshold 0.010)  HIGH_DIFF (cluster medium)
pHash dist: 8      (threshold 4)      HIGH_DIFF
Layout:     0.94    (threshold 0.98)   HIGH_DIFF
RESULT: HIGH_DIFF
BUNDLE: .\reports\qa\vp\home-light.vpbundle
EXIT_CODE: 2
```

Escenario 2 — Batch de un directorio entero

```
PS> visualparity batch `
    --canon-dir .\qa\_mockup_canonical `
    --actual-dir .\reports\qa\actual `
    --out-dir   .\reports\qa\vp `
```

```
--profile    strict

# STDOUT summary (parseable por agentes):
Processed: 116/116
AUTO_HIGH_DIFF:    40
AUTO_SUSPICIOUS:   37
LOW_DIFF:           29
NO_DIFF:            6
SIZE_MISMATCH:     4
RESULT: HAS_HIGH_DIFF
BUNDLE_DIR: .\reports\qa\vp
EXIT_CODE: 2
```

Escenario 3 — Inspeccionar un bundle existente

```
PS> visualparity inspect .\reports\qa\vp\home-light.vpbundle

Bundle: home-light.vpbundle
Generated: 2026-07-07T10:15:30Z
Surface: suite:home@light
Profile: strict
State:    HIGH_DIFF

Metrics:
  SSIM:      0.987
  MS-SSIM:   0.992
  Changed:   0.042 (1 cluster medium)
  pHash dist: 8
  Layout:    0.94

Determinism: PASS
State assertion: VALID
Capture provenance:
  Commit: abc123d...
  capture_v8.py sha256: 9f8e7d6c...
  Qt version: 6.6.0

Findings:
  [HIGH] REGIONAL_DIFF: 1 cluster medium en bbox [120,80,60,30]
```

Escenario 4 — Exportar a HTML report

```
PS> visualparity export `
    --bundle-dir .\reports\qa\vp `
    --format      html `
    --out         .\reports\qa\report.html

# Genera un único archivo HTML con:
# - Index navegable de 116 surfaces
# - Página por surface con CANON/ACTUAL/DIFF embebidos (base64)
# - Métricas en tabla, severity badge
# - Filtros por family/theme/state
# Output: report.html (aprox 5-15 MB dependiendo del número de surfaces)
```

Escenario 5 — nm suite harness invoca VisualParity

```
# harness/ci_gate/gate.py – V3 (extracto)

import subprocess, json, sys

from harness.bundle_loader import load_and_verify
from harness.policy.loader import load_policy, evaluate

def run_gate(surface_key: str, canon_dir: str, actual_dir: str) -> str:

    # 1. invocar VisualParity compare

    bundle_path = f"reports/qa/vp/{surface_key.replace(':', '-').replace('@', '-')}.vpbundle"

    canon = f"{canon_dir}/{surface_key.split('@')[0].replace(':', '/')}-{surface_key.split('@')[1]}.png"

    actual = f"{actual_dir}/{surface_key.split('@')[0].replace(':', '/')}-{surface_key.split('@')[1]}.png"

    r = subprocess.run([
        "visualparity", "compare",
        "--canon", canon,
        "--actual", actual,
        "--out", bundle_path,
        "--profile", "strict", # hardcoded en policy
        "--surface-key", surface_key,
        "--determinism-check",
    ], capture_output=True, text=True)

    if r.returncode == 4:

        return f"BLOCK: visualparity_error: {r.stderr}"

    # 2. cargar + verificar bundle
```



```
bundle = load_and_verify(bundle_path)

# 3. anti-fraud scan

anti_fraud_ok = subprocess.run([sys.executable, "harness/anti_fraud/scan.py", "--mode", "all"]).returncode == 0

# 4. cargar policy y evaluar

policy = load_policy("harness/policy/closure_policy.yaml")

verdict = evaluate(policy, bundle, anti_fraud_ok=anti_fraud_ok)

return verdict.action # ALLOW_CLOSURE | BLOCK | HUMAN_REVIEW | BLOCK_OWNER_DECISION
```

6.5 Diagrama de flujo completo del sistema

Flujo end-to-end — del agent_runner al cierre reproducible

VisualParity V3 — Propuesta técnica

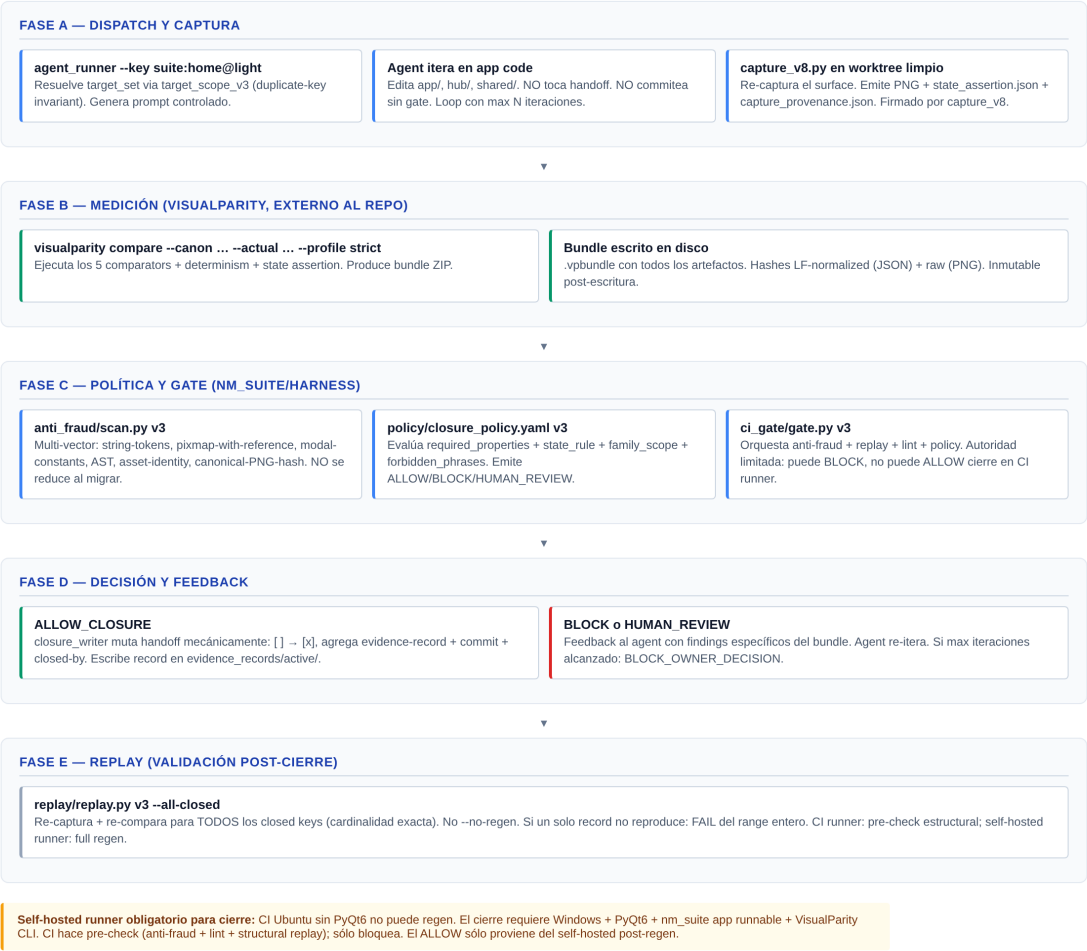


Figura 6.1 — Flujo end-to-end del sistema V3. Del agent_runner al cierre reproducible. Self-hosted runner obligatorio para cierre; CI sólo bloquea. Replay re-captura + re-compara siempre.

El flujo completo: agent_runner recibe --key suite:home@light → target_scope v3 resuelve (fail-fast si duplicate) → capture_v8 re-captura en worktree limpio → VisualParity produce bundle → anti_fraud v3 escanea producto (9 vectores) → policy v3

evalúa bundle + anti-fraud → ci_gate v3 decide (BLOCK en CI, ALLOW en self-hosted) → si ALLOW, closure_writer muta handoff mecánicamente y escribe evidence record → replay v3 re-valida al commit con re-captura + re-comparación. Cualquier fallo en cualquier etapa BLOCK; no hay fallback.

7. Conclusión y recomendación final

El diagnóstico es inequívoco. V1 fue fraude sistémico: tres razones técnicas independientes (hashes CRLF no reproducibles, replay estructural en CI sin regen, closer que no parsea el texto del handoff) invalidaron los 116/116 closures declarados PASS. V2, lejos de corregir V1, lo agravó con stubs que PASS, anti-fraud reducido al 11% de cobertura, auto-inconsistencia (su scope resolver y su doc lint fallan en su propio estado commiteado), y dependencia de VisualParity que no existe.

V3 es el primer diseño que respeta la invariante fundamental: **VisualParity mide y muestra. El harness decide**. Esta separación estricta entre medición y política es lo que V1 y V2 violaron sistemáticamente. V1 mezcló medición + política + escritura en un solo binario. V2 declaró la separación pero la implementó con stubs vacíos y datos de medición hardcoded en la política. V3 codifica la separación como invariantes de diseño: VisualParity no tiene dependencias hacia el repo consumidor, no conoce el formato del handoff, no conoce closure_policy.yaml. El único contrato entre VisualParity y el harness es el formato del evidence bundle.

Las 22 reglas técnicas derivadas de la auditoría forense (sección 5 del Anexo) son implementables en 5 fases (19-26 semanas). La Fase 1 (CLI/Core) es la más crítica: sin VisualParity funcional, el resto del sistema no puede empezar. La Fase 4 (integración nm_suite) es la más larga por la migración de los 116 closures V1 inválidos. La Fase 5 (agent workflows) es la que cierra el loop y permite trabajo autónomo de agentes sin intervención humana para la mayoría de surfaces.

Recomendación final

(1) **No reutilizar V1 ni V2** salvo assets canónicos (PNGs, HTML, receta) y utilitarios limpios (vas_engine, vas_introspect, capture_v8 con extensión). (2) **Re-abrir los 116 closures V1 como OPEN** en el handoff V3; tratar como deuda visual pendiente. (3) **Empezar Fase 1 inmediatamente**: la CLI de VisualParity es el bloqueante crítico. (4) **Eliminar --no-regen** y reopen_legacy_all del harness; son las dos superficies más peligrosas de V1. (5) **Self-hosted runner para cierre**; CI sólo bloquea. (6) **Audit log inmutable** para todas las decisiones humanas; append-only JSONL con reviewer_id + bundle_hash + rationale.

Anexo A — Glosario técnico

Términos clave usados en este documento. El glosario es referencia rápida; para definiciones formales, ver el schema del evidence bundle y el closure_policy.yaml v3.

Término	Definición
surface_key	Identificador único de una pantalla/estado visual. Formato: {familia}-{subfamilia}@{theme}. Ej: suite:timer-running@light.
family	Agrupación de surfaces relacionadas. Ej: timer agrupa timer-running, timer-paused, timer-empty.
theme	Variante visual de la misma surface. Ej: light, dark.
evidence bundle	ZIP inmutable (.vpbundle) producido por VisualParity. Contiene canonical/actual/diff PNGs + metrics + state_assertion + capture_provenance + firma.
evidence record	JSON en harness/evidence_records/active/ escrito por closure_writer tras ALLOW_CLOSURE. Referencia al bundle + commit + reviewer.
AUTO_PASS	Señal de medición emitida por VisualParity. NO es autoridad de cierre.
CLOSURE_PASS	Autoridad de cierre emitida por el harness tras ALLOW_CLOSURE. Requiere AUTO_PASS + required_properties OK.
HUMAN_REVIEWED_PAS S	Decisión humana registrada para una surface SUSPICIOUS. Requiere reviewer_id + rationale + bundle_hash en sidecar JSON.
canonical	PNG de referencia en qa/_mockup_canonical/. Fuente de truth visual.
actual	PNG producido por capture_v8.py capturando la app en runtime.
diff	PNG con pixels changed resaltados, producido por VisualParity.
contact sheet	PNG mosaico con miniaturas de múltiples surfaces ordenadas por severidad. Útil para revisión humana en pared.
SSIM	Structural Similarity Index. Métrica [0,1] que mide similitud estructural entre dos imágenes.
MS-SSIM	Multi-Scale SSIM. Aplica SSIM a múltiples escalas para capturar diferencias a varias frecuencias.
PixelMatch	Comparador pixel-a-pixel con threshold perceptual (alpha < 0.1) para descartar diferencias sub-pixel.
pHash	Perceptual Hash. DCT-based hash 64-bit. Distancia Hamming detecta layout shifts.
bbox clustering	Agrupación de pixels changed en regiones conectadas. Top-N clusters por área con severity.
layout diff	Comparador estructural. Edge-detection + connected-components. Detecta elementos faltantes o extraños.
state assertion	JSON firmado por capture_v8.py con el estado del widget al momento de captura. Verifica que la captura corresponde al surface_key.
determinism check	Dos captures del mismo surface sin commit intermedio. Si inter_capture_ratio ≥ 0.005 , flag non_deterministic.
replay full-regen	Re-captura + re-comparación de todos los closed keys. Único modo de replay en V3.

Término	Definición
replay structural	Modo de replay V1 que NO re-captura. Eliminado en V3.
anti-fraud multi-vector	9 categorías de check: string-tokens, pixmap-ref, modal-constants, AST, asset-identity, canonical-PNG-hash, capture-script-tamper, sidecar-provenance, stale-bundle.
near-threshold	Flag emitido cuando una métrica está dentro del 5% del threshold. Dispara HUMAN_REVIEW.
OWNER_EXCEPTION_ACTIVE	Reemplazo V3 de DECISIÓN-OWNER. Schema estricto con rationale + reviewer_id + bundle_hash. Append-only audit log.
target-set	Lista de surface_keys a procesar en una sesión de agente. Resuelto por target_scope.py v3 con duplicate-key invariant.
family-scope	Regla que bloquea cierre parcial de familia sin OWNER_EXCEPTION_ACTIVE.
reward hacking	Comportamiento del agente que maximiza PASS sin hacer el trabajo real. V3 mitiga via 10 vectores.
stale bundle	Bundle con generated_at anterior al último commit. BLOCK en V3.
CRLF/LF normalization	Conversión de \r\n a \n antes de hashear archivos de texto. NO aplica a binarios (PNG).

Anexo B — Resumen de las 38 hipótesis red-team

Tabla completa de las 38 hipótesis red-team del usuario contrastadas contra el código real del repo. Status: ☒ confirmada, ☐ parcial (no ejecutable en sandbox), ☐ ? requires deeper check (subproducto), ☐ refutada. Cero hipótesis falsas.

#	Hipótesis	Status
1	Bug CRLF/LF afecta los 116 records activos	✓
2	Hashes dependientes del checkout Windows, no aleatorios	✓
3	Evidence byte-perfect ≠ comparator PASS	✓
4	Fuentes concretas de no determinismo (actividades, respiración, rutina-add-task)	⚠
5	target_scope.py protege/excluye keys cerradas	✓
6	close_visual_key.py no cierra key ya cerrada salvo reapertura	✓
7	replay_visual_closure.py no revalida todo el historial cerrado	✓
8	Replay estructural puede pasar mientras replay full falla	✓
9	Las keys con replay full FAIL son 11	?
10	La emulación reversible logró cerrar 116/116 por script	✓

#	Hipótesis	Status
11	La emulación reversible NO produjo cierre reproducible 116/116	?
12	Las 11 keys fallidas pasaron run_visual dos veces	?
13	Causa de las 11: captura inestable (9) + report volátil (2)	⚠
14	El gate visual actual no mostró FAIL visual oculto en 116	X (refutada por V2)
15	Hay falso PASS visual actual o falso PASS comparator confirmado	✓
16	timer-running@light puede pasar comparator usando estado equivocado	✓
17	Mutaciones visuales visibles pueden pasar comparator	✓
18	Closer puede cerrar key con semántica activa de FAIL/deuda	✓
19	Closer puede dejar contradicciones semánticas en el handoff	✓
20	replay_visual_closure.py puede devolver PASS vacío	✓
21	Parser/handoff permite spoofing por key duplicada	✓
22	Anti-fraud aborta pero puede dejar reporte PASS viejo en OutDir	✓
23	Canonical/reference smuggling obvia fue bloqueada	✓
24	Fuentes no activas producen REPORT_EVIDENCE_VALID:NO	✓
25	VAS no acepta sidecar de key equivocada	✓
26	Modal/backdrop alterado falló comparator y modal audit	✓
27	Tamper básico de evidence es detectado	✓
28	reopen_legacy_all es superficie peligrosa de gobernanza	✓
29	Tests de gate están parcialmente stale	⚠
30	DECISIÓN-OWNER era la palabra clave correcta del gap	✓
31	Gap semántico de DECISIÓN-OWNER era real	✓
32	Hardening local canon-first corrige gap pero no retroactúa	✓
33	Nuevo protocolo prohíbe usar riesgo/costo/"requiere decisión" como excusa	✓
34	Aún queda inconsistencia activa en DECISIÓN-OWNER	✓
35	Hay cierres actuales con metadata visible de divergencia alta aunque tengan evidence	✓
36	Hay near-miss PASS muy cercanos al umbral	✓
37	Hay gap entre política family/scope y enforcement real del closer	✓
38	El cierre histórico 116/116 no puede tratarse como homogéneo	✓

Resumen: 30 ☒ confirmadas con citas file:func:line, 4 ☐ parciales (no ejecutables en sandbox por falta de PyQt6/libEGL), 3 ? requires deeper check (subproductos de las parciales), 1 ☐ refutada (la hipótesis #14, refutada por V2 mismo). **Cero hipótesis**

falsas. La validación forense completa con evidencia código está en `/home/z/my-project/worklog.md` sección FORENSIC-AUDIT-1.